

# 6.smolvla真机部署

---

## 一、测试与lerobot适配的几个脚本

### 1.硬件测试和lerobot数据集格式适配

#### ( 1 ) lerobot数据集格式

##### 1.任务指令从哪里输入？

### 2.数据采集脚本弄懂

#### ( 1 ) 数据质量监控类

##### 1.四个维度的数据质量的评价指标

##### 2.怎么实现跳过帧的？触发条件是什么？

##### 2.1 主线程中的数据录制逻辑

#### ( 2 ) 相机线程是怎么工作的？

##### 1.怎么理解python中这个线程的创建和线程锁的作用？

##### 1.1 with self.lock到底啥意思？

##### 1.2 with self.lock: 到底做了什么？

##### 1.3 如果不加锁会怎么样？

##### 1.4 self.thread.join(timeout=2)是啥意思？

##### 2.相机线程是独立于主线程的吗？其中的loop循环频率是多少？

##### 1. threading.Thread(...)

##### 2. target=self.\_loop

##### 3. daemon=True

4.thread.join()

### ( 3 ) 主控制器类

1.键盘控制机械臂伺服增量运动是怎么实现的？

2.是怎么实现夹爪的非阻塞控制的？

3.是怎么实现夹爪在闭合时读取位置延迟的？怎么进行线性插值的？

4.录制状态机是怎么工作的？

4.1 状态设置

4.2 状态转移

5.键盘监听的工作原理

keyboard.Listener 到底做了什么？

1. 创建一个后台线程

2. 绑定回调函数

3. 不断监听系统键盘事件

4. 事件来了就调用你给的函数

start() 做了什么？

6.怎么实现数据的异步保存（写盘），不阻塞主线程的机械臂控制

7.为什么不把10hz的数据录值的指令发送也放到数据写入后台中呢？

8.

### ( 4 ) 数据录制线程

1.是怎么实现数据的异步保存数据，不阻塞控制的？

1.1 datasert\_queue.get(timeout=0.1)是什么

任务 1：add → 录一帧数据

任务 2：save → 保存完整一段数据

任务 3 : clear → 废弃不要的数据

任务 4 : finalize → 关闭数据集

2. 怎么实现任务指令的随机抽取的 ?

( 5 ) 在结束程序时各个线程是怎么清理资源的 ?

1. finally 的作用

2. 清理资源时最后 finalize 的作用是什么 ?

3. 最终版异步推理部署脚本

9. 离线测试模型推理结果是否合理的脚本 ( 同步推理 )

1. 脚本的作用

2. 如何使用这个脚本 ?

参数解释 ( 你只需要改这 3 个 )

3. 使用时的注意事项

( 1 ) 传入的训练模型路径

( 2 )

3. 对脚本的一些疑问

2.1. 不连机械臂, 模型到底怎么推理 ?

2.2. “机械臂状态” 是哪来的 ?

2.3. 动作输出多少步, 由谁决定 ?

2.4. 这个脚本的作用是不是 : 分析模型输出的动作是否合理 ?

4. 脚本的代码

10. 在线部署到法奥3上的推理脚本(同步推理)

11. 在线部署到法奥3上的推理脚本(异步推理, 性能更优)

( 1 ) 之前脚本的推理方式

( 2 ) SmolVLA的动作块(Action Chunk)机制:

( 3 ) 异步推理：终极方案

## 12.最终使用版本的异步推理脚本理解

( 1 ) 程序架构

模块 1：摄像头线程 CameraThread

模块 2：AI 推理线程 InferenceThread ( 最核心 )

推理线程执行流程：

模块 3：主线程 ( 主循环 50~100Hz )

完整程序执行流程

【阶段 1：初始化】

【阶段 2：推理线程第一次跑】

【阶段 3：主线程开始控制循环】

【阶段 4：退出】

( 2 ) 对于action\_chunk的时间集成机制是怎么做的？有什么效果？为什么叫时间集成机制？

1.实现方式

1.1 触发时机

1.2 重叠融合核心逻辑 ( 对于重叠部分可以用线性加权平均做平滑，也可以用指数平滑 )

1.3 权重设计 ( 线性插值 )

1.4 权重设计 ( 指数加权平滑 )

( 3 ) 如果主线程的消费频率过快，为什么会使得动作执行抖动？或者出现了回退运动的现象？

1.在什么情况下会出现队列空的情况？和什么有关？

2.如果出现了空队列的情况，机械臂在实际执行中会有什么样的效果？

( 4 ) 推理线程中模型输入数据类型是什么样的？

#### 1.observation.images.top

- ① frame\_rgb ( 推理前 )
- ② torch.from\_numpy(frame\_rgb)
- ③ .permute(2, 0, 1) 【核心！】
- ④ .unsqueeze(0)
- ⑤ .float()
- ⑥ .to(self.device)
- ⑦ / 255.0

#### 2.observation.state

#### 3.关键的一些转变

#### 4.最终输入模型的数据格式

( 5 ) 为什么推理时一定要加 policy.eval() ? 不加会怎么样 ?

- 1.eval() 到底关闭了什么 ?
- 2.如果 不加 policy.eval () 会发生什么 ?
- 3.超简单比喻 ( 秒懂 )

( 6 ) 夹爪的控制逻辑是怎么样的 ? 双阈值滞回 , 和连续确认机制

## 二、法奥数据采集

### 1.采集过程

- ( 1 ) 官方建议 :
- ( 2 ) 采集示例 ( 单任务 )
- ( 2.5 ) 模型泛化数据采集

泛化体现的角度 :

1. 物体位置泛化 ( 最重要 )

2. 机械臂起点泛化

3. 视觉泛化 ( 光线、背景、角度 )

4. “自然小失误 + 修正”

具体采集建议

任务 1 : put the cube into the box ( 30 条 )

任务 2 : put the cup on the cup holder ( 30 条 )

( 3 ) 采集步骤 ( 单任务 )

步骤 1 : 准备环境

步骤 2 : 启动程序

步骤 3 : 开始录制 ( 每个 episode 都一样 )

① 机械臂回到初始姿势 → 按 空格 开始录制

② 控制机械臂完成完整动作

③ 动作完成 → 再按 空格 保存

④ 重复上面流程 25 ~ 100 次

( 4 ) 采集步骤 ( 多任务 )

1. 程序修改的地方

2. 采集步骤

开始录制任务 A

修改 task , 录制任务 B

3. 最终数据集结构

( 5 ) 采集必须遵守的 6 条规则

1. 每个 episode 必须完整 !

2. 动作要慢、平稳

3. 方块位置每次稍微变化一点

4. 光线保持一致

5. 不要抖动、不要急停

6. 初始姿势每次尽量一致

## 2.数据集格式 ( lerobot V3.0 )

(1)chunk-000 到底是什么？

1. 为什么要分 chunk？( 为什么不存成一个大文件？ )

2. chunk-000 到底是什么意思？

3. 一个 chunk 里包含什么？

4. 对应到你的目录结构

5. 你录少量数据时 ( 比如 10 ~ 30 条 episode )

6. 超通俗比喻

## 3.数据集采集具体方案设计

## 4.关于 $\pi_0$ 能否直接使用你采集的数据

## 三、法奥数据微调训练记录

### 1.云服务器训练环境创建

- 1.创建文件夹

- 2.创建虚拟环境

- 3.克隆lerobot仓库

- 5.安装lerobot的依赖

- 6.登录wandb

- 7.安装 SmoVLVA 依赖项

- 4.安装ffmpeg

## 2.微调训练的命令

1. --policy.type vs --policy.path 的区别
2. --batch\_size=64 — 可能OOM
3. --steps=20000 — 对50条数据偏多
- 4.学习率设置
- 5.LeRobot 训练脚本的命令行参数不支持 False 字符串，必须用 false 小写！
- 6.数据集的相机名称和smolvla默认相机名称不匹配

## 3.怎么看我的模型训练是否陷入过拟合？

（1）【看 loss】训练 loss 极低，但不再下降，反而开始上升

为什么这种情况是过拟合？

阶段 1：loss 快速下降

阶段 2：loss 降到很低，但不再下降

阶段 3：loss 开始上升（关键！）

为什么这会让 loss 上升？

（2）【看 loss】训练 loss 非常低（0.01），但推理时机械臂完全不动 / 乱动作

## 4.微调时预训练模型加载方式的影响！

（1）发现问题

（2）问题解决和分析

1. 第一种正确的加载模型的方式内部的工作过程

2. 第二种错误的模型加载方式

## 四、真机部署遇到的一些问题

1.什么是伺服关节运动？servoJ和moveJ的区别？

1.1什么是 ServoJ 伺服关节运动？



✅ ServoJ = 实时、高频、连续的关节伺服控制

## 1.2 ServoJ ( 伺服运动 ) vs 普通点到点运动 ( P2P )

1 ) 普通点到点运动 ( MoveJ / 点动 JOG )

2 ) ServoJ 实时伺服运动

## 1.3 最直观的比喻

普通点到点运动 ( MoveJ )

ServoJ 实时伺服

## 1.4 为什么 LeRobot 必须用 ServoJ ?

2.抖动！直接使用sveroJ进行键盘控制机械臂非常抖动！和我在moviet混合规划真机部署时遇到的问题一模...

( 1 ) 为什么直接使用servoJ会出现异常抖动的现象？

( 2 ) 如何解决？

新版解决思路：

初始化 只读取一次真实位置

运动计算 完全基于纯数学

不再读取实时关节用于控制

( 3 ) 这种软件开环控制会不会导致误差累积？

软件开环位置控制 + 机器人底层硬件闭环

LeRobot 所有官方控制方案，全都是这种 “开环发目标” 方式！

行业标准做法：

## 3.夹爪的运动超时问题

## 4.录制数据集时，机械臂控制卡顿问题

卡顿的原因是：

解决方法：

## 5.对smolvla模型的输出动作序列的步数有疑问

- ( 1 ) 模型推理输出的动作块到底是多少步？怎么设定？
- ( 2 ) 在调用select\_action()时，输出的为什么是一个动作？

## 6.踩坑！微调训练命令中训练模型的加载方式的影响（详情见三、微调训练的4）

## 7.增量训练的一些坑！！

- ( 1 ) 问题所在
- ( 2 ) 原因分析
- ( 3 ) 验证分析

## 8.消费频率调高后，运动抖动并且动作回退的现象

## 五、第一次微调训练实机部署记录

### 5.1 第一次微调smolvla并部署到法奥真机上的效果

- ( 1 ) 直接部署效果
  - 1.机械臂非常抖动的原因
  - 2.夹爪开合预测有问题
  - 3.模型成功率很低（ 0% ）
- ( 3 ) 第一次微调数据采集的情况（ 有问题 ）
- ( 4 )

## 六、VLA模型持续学习与泛化增强方案

- 1.核心原则
- 2.具体实施方案
  - 2.1 采集方案总览
  - 2.2 阶段二：提升空间泛化能力的数据采集方案

[2.3 数据集融合的方法（脚本）](#)

[2.4 继续训练的命令（更小的学习率是关键）](#)

和第一次训练的关键区别：

[2.5 阶段三：加入新任务](#)

[（1）数据采集方案](#)

[（2）合并新旧数据集](#)

[3.防止模型遗忘的核心策略](#)

[4.训练后的验证流程](#)

[5.完整的继续训练路线图](#)

## 一、测试与lerobot适配的几个脚本

### 1.硬件测试和lerobot数据集格式适配

#### （1）lerobot数据集格式

[https://blog.csdn.net/m0\\_63007797/article/details/154176818](https://blog.csdn.net/m0_63007797/article/details/154176818)（详细介绍）

最终保存的文件组织形式：

▼ Plain Text

```
1  dataset_root/
2  |— meta/                                # 元数据目录
3  |   |— info.json                       # 数据集schema信息
4  |   |— stats.json                      # 全局特征统计（归一化用）
5  |   |— tasks.parquet                   # 任务描述映射
6  |   |— episodes/                       # Episode元数据（分块存储）
7  |       |— chunk-000/
8  |           |— file-000.parquet        # Episode记录（长度、任务、偏移）
9
10 |— data/                                # 表格数据目录
11 |   |— chunk-000/
12 |       |— file-000.parquet            # 多个episode的帧数据
13
14 |— videos/                              # 视频数据目录
15 |   |— front_camera/                   # 按摄像头分目录
16 |       |— chunk-000/
17 |           |— file-000.mp4            # 多个episode的视频帧
18 |   |— wrist_camera/
19 |       |— chunk-000/
20 |           |— file-000.mp4
21
```

程序中数据的存储特征：



Python

```
1  # 定义数据集格式 (图像+关节状态+动作+任务指令)
2  features = {
3      "observation.images.top": {
4          "dtype": "video",
5          "shape": (IMG_SIZE, IMG_SIZE, 3),
6          "names": ["height", "width", "channel"],
7      },
8      "observation.state": {
9          "dtype": "float32",
10         "shape": (7,),
11         "names": self.STATE_NAMES,
12     },
13     "action": {
14         "dtype": "float32",
15         "shape": (7,),
16         "names": self.STATE_NAMES,
17     },
18 }
19 # 创建LeRobot数据集
20 self.dataset = LeRobotDataset.create(
21     repo_id=self.repo_id,
22     fps=self.record_fps,
23     robot_type="fa3",
24     features=features,
25     use_videos=True,
26 )
```

## 1.任务指令从哪里输入？

```
print(f" ⚠ 已静止 {idle_sec:.1f}秒，帧被跳过中... (请继续操作)")

if should_record:
    # 加入数据集（主线程这里并不直接写入数据盘，而是通过队列缓存的方式发给后台专门处理写盘的线程进行处理。）
    self.dataset_queue.put({
        "cmd": "add",
        "frame": frame_ref,
        "state": state_7d,
        "action": action_7d,
        "task": self.current_episode_prompt #按空格时，随机抽取的任务指令
    })
    self.quality_monitor.on_frame_recorded(state_7d, action_7d)
else:
    # 跳过静止帧
    self.quality_monitor.on_frame_skipped()
```

- 从队列缓存中

## 2.数据采集脚本弄懂

### （1）数据质量监控类

#### 1.四个维度的数据质量的评价指标

- 1.录制时长，15~25秒最佳
- 2.有效帧率：跳过率越低越好
- 3.夹爪变化：至少2次（开→闭→开）

- 4.关节运动幅度：前6个关节都有足够运动

## 2.怎么实现跳过帧的？触发条件是什么？

定义了一个检测机械臂或者夹爪是否在运动的函数：

Python

```
1     def _is_moving(self):
2         """检测机械臂是否在运动：键盘按下 或 夹爪运动"""
3         with self.lock:
4             keys = set(self.pressed_keys)
5             has_key = any(k in self.CART_KEY_MAP for k in keys)
6             return has_key or self.gripper_moving_state
```

触发条件就是：连续的0.5s没有机械臂或者夹爪的运动（通过静止计数器），录制标志位就置false

```
if currently_moving:
    # 有运动：重置静止计数，正常录制
    self.idle_frame_count = 0
    should_record = True
else:
    # 无运动：静止计数+1，超过阈值则跳过
    self.idle_frame_count += 1
    should_record = (self.idle_frame_count <= self.max_idle_frames)
```

```
if should_record:
    # 加入数据集（主线程这里并不直接写入数据盘，而是通过队列缓存的方式发给后台专门处理写盘的线程进行处理。）
    self.dataset_queue.put({
        "cmd": "add",
        "frame": frame_ref,
        "state": state_7d,
        "action": action_7d,
        "task": self.current_episode_prompt #按空格时，随机抽取的任务指令
    })
    self.quality_monitor.on_frame_recorded(state_7d, action_7d)
else:
    # 跳过静止帧
    self.quality_monitor.on_frame_skipped()
```

后面就根据录制标志位进行录制，如果超过静止时间，就跳过这一帧，不进行录制。

假设我操作静止了1s，那我只会录制前0.5s，后面的就不会录制，除非又开始运动了。

## 2.1 主线程中的数据录制逻辑

只有在按下空格之后，才会每0.1s采集一次数据。



Python

- 1 正在录制时，每 0.1 秒（10Hz）采集一帧数据 →
- 2 检查机械臂是否在动 →
- 3 动就录，不动超过 0.5 秒就自动跳过不录，避免垃圾数据

## （2）相机线程是怎么工作的？



## 1. 怎么理解python中这个线程的创建和线程锁的作用？

### 1.1 with self.lock到底啥意思？

**作用：**防止多个线程同时访问同一个变量，造成程序的崩溃。通过给这个变量加锁的方式，告诉其他所有的线程当前这个变量正在被这个线程访问，你们先停止访问。

你可以把 `self.lock` 理解成：

 一个厕所的门锁

- 同一时间，只能有一个人用
- 别人要用，必须等你用完出来

代码里的逻辑：

```
Python |  
1  with self.lock:  
2      keys = set(self.pressed_keys)
```

翻译成成人话就是：

**“我现在要独占使用 `pressed_keys`，谁都不许动，等我读完你们再继续。”**

### 1.2 with self.lock: 到底做了什么？

我拆开给你看：

```
▼ Python |  
1  with self.lock:  
2      keys = set(self.pressed_keys)
```

等于：

### 1. 上锁

- 告诉所有线程：
- “这个资源我承包了，你们都等着！”

### 2. 执行代码

- 安全地读取按键
- 别人无法修改

### 3. 自动解锁

- 退出 `with` 语句时**自动解锁**
- 其他线程可以继续访问

## 1.3 如果不加锁会怎么样？

```
▼ Python |  
1  # 这是在主线程里的操作，主线程想要访问pressed_keys变量，但是键盘监听线程还在不断的写这个变量  
2  with self.lock:  
3      keys = set(self.pressed_keys)  # 读按键
```

### 【最坏情况】

1. 主线程 **正在读** `pressed_keys`
2. 同时，键盘线程 **正在改** `pressed_keys`
3. 读读到一半，集合被改了 → **程序直接报错崩溃！**

这就叫：

### 线程竞争 ( Race Condition )

▼ Plain Text

```
1  主线程：正在复制 {w,a,s}
2  键盘线程：突然删掉 a → 变成 {w,s}
3  主线程：复制到一半，集合变了 → 程序直接崩！
```

这就是为什么必须加锁。

#### 1.4 `self.thread.join(timeout=2)`是啥意思？

### 2.相机线程是独立于主线程的吗？其中的loop循环频率是多少？

- 是独立于主线程

▼ Python

```
1  self.thread = threading.Thread(target=self._loop, daemon=True)
2  #
3  我要创建一个新的后台线程，
4  让它去跑 self._loop() 这个函数，
5  并且把它设置为【守护线程】，
6  最后把这个线程存到 self.thread 里方便管理。
```

### 1. `threading.Thread(...)`

创建一个线程对象相当于：“我要开一个新的子任务 / 小助理”

### 2. `target=self._loop`

最重要的参数：线程要跑哪个函数？

- `target` = 你要在线程里跑的函数
- `self._loop` = 要跑的函数（比如相机循环取图、键盘监听循环）

👉 意思：线程启动后，会自动调用 `self._loop()`

### 3. `daemon=True`

设置为【守护线程】（后台线程）

这是超级关键的知识点：

守护线程 = 主线程死，我也死

- 主线程（主程序）退出
- 所有 `daemon=True` 的线程自动跟着退出
- 不用手动关闭，非常安全

如果不写 `daemon=True` 会怎样？

- 程序关了
- 子线程还在后台跑

- 程序**关不掉**，必须强制杀进程

👉 `daemon=True` 就是为了让程序退出干净！

#### 4.thread.join()

“我（主线程）就站在这儿不动了，等你（子线程）彻底跑完、下班了，我再继续走！”

详细解释：

1. 调用 `join()` 的线程（通常是**主线程**）会**暂停执行**，被阻塞住。
2. 一直等到 `self.thread` 这个子线程**彻底运行结束**（从 `_loop` 函数里退出来）。
3. 子线程结束后，`join()` 才会放行，主线程继续往下走。

作用：

**确保子线程干完活，主线程才收尾。**比如：关闭相机前，确保相机线程已经停止读取



Python

```
1  # 1. 创建线程（助理就位）
2  self.thread = threading.Thread(target=self._loop, daemon=True)
3
4  # 2. start() — 启动后台读图（助理开始干活）
5  self.thread.start()
6
7  # ...（主线程继续跑 100Hz 控制，互不干扰）
8
9  # 3. 程序退出时，stop() 函数里
10 def stop(self):
11     self.running = False # 告诉循环停止
12     self.thread.join(timeout=2) # 等线程 2 秒，确保它真的退出了
13     self.cap.release()
```

- `start()` → 相机线程**后台疯狂读图**。
- 主程序**流畅控制机器人**。
- 退出时 → `join()` → **主线程耐心等待相机线程彻底死掉**，再释放相机资源。

- loop循环频率没有设置，能跑多快跑多快，受限于**相机本身的帧率**（你设置的是 30 FPS）

▼ Python

```
1  def _loop(self):
2      while self.running: # 无限循环
3          ret, frame = self.cap.read() # 从相机读一帧
4          if ret:
5              with self.lock:
6                  self.latest_frame = frame
```

你在代码里写了：

▼ Python

```
1  self.cap.set(cv2.CAP_PROP_FPS, 30)
```

所以：

相机最多只能输出 30 帧 / 秒

→ `cap.read()` 最多执行 30 次 / 秒

→ `_loop` 循环速度  $\approx 30\text{Hz}$ （30 次 / 秒）

如果相机没设置 FPS , 它会**跑满速** ( 可能 100+ Hz ) 。

## ( 3 ) 主控制器类

### 1. 键盘控制机械臂伺服增量运动是怎么实现的 ?

在**夹爪不动**的时候 , 读取你按的**键盘** → 计算出机械臂末端应该走多少距离 → 发给**机器人** → 让它平滑移动全程 **100Hz 高频控制** , 所以手感丝滑。



Python

```
1  # --- 3. 笛卡尔空间增量伺服控制 ---
2  if not self.gripper_moving_state:
3      with self.lock:
4          keys = set(self.pressed_keys)
5
6      # 6轴增量初始化为0
7      delta = [0.0] * 6
8      is_moving = False
9      # 根据按键计算位移/旋转增量
10     for k in keys:
11         if k in self.CART_KEY_MAP:
12             dim_idx, direction = self.CART_KEY_MAP[k]
13             if dim_idx < 3:
14                 # 平移轴: mm
15                 delta[dim_idx] += direction * self.linear_speed * dt_contr
16             else:
17                 # 旋转轴: °
18                 delta[dim_idx] += direction * self.angular_speed * dt_cont
19
20     is_moving = True
21
22     # 发送伺服控制指令
23     if is_moving:
24         self.robot.ServoCart(mode=2, desc_pos=delta, cmdT=dt_control)
25     else:
26         # 无按键: 保持当前位置
27         self.robot.ServoCart(mode=2, desc_pos=[0, 0, 0, 0, 0, 0], cmdT=dt_
control)
```



## 2. 是怎么实现夹爪的非阻塞控制的？

用户按 O/C → 只发一次指令 → 不等待 → 主循环继续跑 → 后台自动监控夹爪是否到位 → 到位后自动恢复机械臂控制

Python

```
1 这段代码 = 夹爪状态机
2      1. 空闲状态：等待用户按 O/C
3      2. 触发动作：发指令 → 标记运动中
4      3. 运动中：不等待，主循环继续跑 （读取键盘、发送数据录制指令等，只是停掉了机械臂的伺服
    控制）
5      4. 到位检查：每次循环看一眼
6      5. 到位完成：恢复机械臂 → 回到空闲
7 非阻塞的本质：
8 不等待、不阻塞、不卡死发完命令立刻走，下次循环再检查
```

## 3. 是怎么实现夹爪在闭合时读取位置延迟的？怎么进行线性插值的？

## 4. 录制状态机是怎么工作的？

### 4.1 状态设置

**状态机驱动：** `record_state` 是核心状态变量，取值包括：

- `IDLE`：空闲（无录制）；`RECORDING`：正在录制；`SAVE_PENDING`：待保存；`DISCARD_PENDING`：待废弃。

## 4.2 状态转移

## 5. 键盘监听的工作原理

我是在主线程循环开始的时候启动了键盘监听

```
def run(self):
    """主运行循环：程序入口，调度所有功能"""
    '''「初始化交互→硬件实时控制→数据采集→状态机管理→用户交互」'''
    # 初始化连接
    # 这一步会开启两个后台线程
    self.connect() #1.连接了机械臂；2.启动了相机线程（相机实时采帧）；3.获取了机械臂的各种状态；4.定义数据集格式，创建数据集
    # 启动键盘监听
    listener = keyboard.Listener(on_press=self._on_press, on_release=self._on_release) #非阻塞监听用户输入
    listener.start()

    # 打印操作说明
```

我启动一个独立的后台线程，专门盯着键盘，你（主线程）继续跑你的主循环，不用管我！

`keyboard.Listener` 到底做了什么？

它内部做了 4 件事：

### 1. 创建一个后台线程

1 新线程 = 监听键盘

## 2. 绑定回调函数

| ▼                                       | Plain Text |
|-----------------------------------------|------------|
| 1 按下 → 调用 <code>self._on_press</code>   |            |
| 2 松开 → 调用 <code>self._on_release</code> |            |

## 3. 不断监听系统键盘事件

| ▼                          | Plain Text |
|----------------------------|------------|
| 1 <code>while True:</code> |            |
| 2     等待键盘事件               |            |

## 4. 事件来了就调用你给的函数

你按 `w` → 自动调用 `_on_press('w')`

`start()` 做了什么？

| ▼                               | Python |
|---------------------------------|--------|
| 1 <code>listener.start()</code> |        |

启动后台线程，让它开始跑！

启动后：

- 监听线程开始跑
- 主线程立刻继续执行下一行代码

- 不会等待，不会卡住

## 6.怎么实现数据的异步保存（写盘），不阻塞主线程的机械臂控制

- 在主线程中，只根据录制状态机，通过 `dataset_queue` 向 `_dataset_worker`（异步数据写入线程）发送指令，是保存？还是废除？从而**避免 IO 操作阻塞 100Hz 控制循环**，导致在录制时机械臂卡顿。

## 7.为什么不把10hz的数据录值的指令发送也放到数据写入后台中呢？

8.

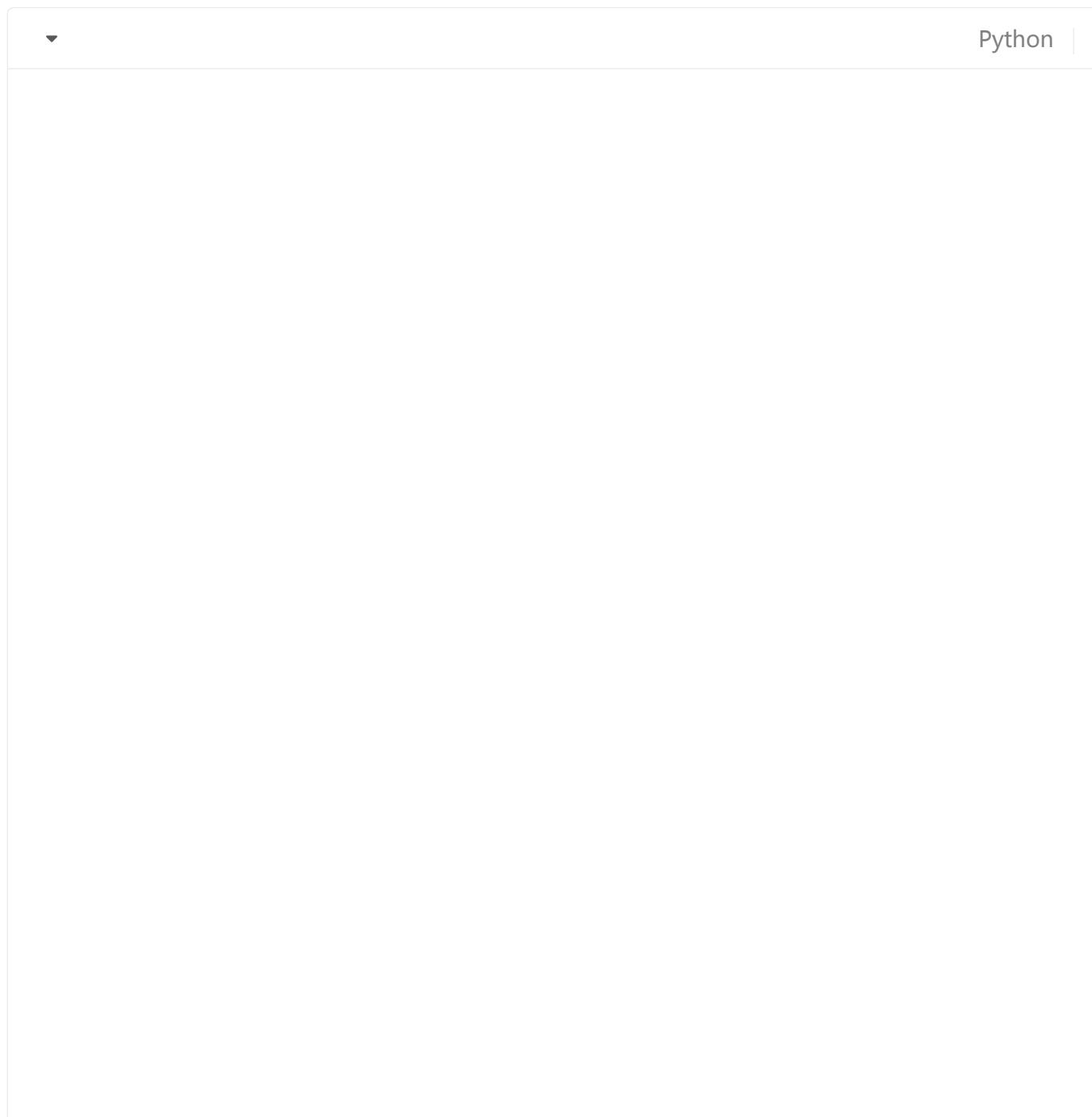
## （4）数据录制线程

### 1.是怎么实现数据的异步保存数据，不阻塞控制的？

```
Python |  
1 self.dataset_thread = threading.Thread(target=self._dataset_worker, daemon=True)
```

在主循环中，每10hz，根据当前的录制状态，如果是正在录制状态，并且没有静止，就会给会向队列里

存入【命令+要存储的信息】，这个命令就是：添加/保存/删除/finalize，也就是主线程只会根据当前状态，把命令和信息存到队列缓存中，并不会做写入磁盘这种耗时操作。而在后台的子线程中，\_dataset\_worker这个函数中，会循环检查队列里存储的命令，进行数据存储操作（ ）



```
1  def _dataset_worker(self):
2      """
3      数据集后台工作线程：
4      处理添加帧、保存、清空、最终化等操作，异步执行不卡顿
5      """
6      while self.running:
7          try:
8              msg = self.dataset_queue.get(timeout=0.1)
9              cmd = msg["cmd"]
10
11             if cmd == "add":
12                 # 缩放图像并转为RGB格式
13                 frame_resized = cv2.resize(msg["frame"], (IMG_SIZE, IMG_SIZE))
14                 frame_rgb = cv2.cvtColor(frame_resized, cv2.COLOR_BGR2RGB)
15                 # 添加一帧数据到数据集
16                 self.dataset.add_frame({
17                     "observation.images.top": frame_rgb,
18                     "observation.state": np.array(msg["state"], dtype=np.float32),
19                     "action": np.array(msg["action"], dtype=np.float32),
20                     "task": msg["task"],
21                 })
22             elif cmd == "save":
23                 # 保存当前episode
24                 self.dataset.save_episode()
25                 self.dataset.clear_episode_buffer()
26                 print(f"\n✅ 已保存！当前数据集总录制：{self.dataset.num_episodes} 条")
27             elif cmd == "clear":
28                 # 清空当前未保存数据
29                 self.dataset.clear_episode_buffer()
30                 print(f"\n❌ 已废弃！当前正在录制的的数据已清空。")
31             elif cmd == "finalize":
```

```
32             # 最终化数据集（写入索引文件）
33             self.dataset.finalize()
34
35         except queue.Empty:
36             pass
37
```

### 1.1 dataset\_queue.get(timeout=0.1)是什么

就是一个**任务排队通道**：

- 主线程：**往队列里丢任务**（ add / save / clear / finalize ）
- 这个线程：**从队列里取任务**，一个一个执行

timeout=0.1

每 0.1 秒看一次有没有新任务，不会卡死。

**任务 1：** add → 录一帧数据

```
1  frame_resized = cv2.resize(msg["frame"], (IMG_SIZE, IMG_SIZE))
2  frame_rgb = cv2.cvtColor(frame_resized, cv2.COLOR_BGR2RGB)
3
4  self.dataset.add_frame({
5      "observation.images.top": frame_rgb, # 相机图片
6      "observation.state": np.array(msg["state"]), # 机器人状态
7      "action": np.array(msg["action"]), # 动作
8      "task": msg["task"] # 任务指令
9  })
```

作用：



## 把一帧完整的数据（图像 + 机器人 + 动作）存进数据集缓存

### 任务 2：save → 保存完整一段数据

```
▼ Python |
1 self.dataset.save_episode()
2 self.dataset.clear_episode_buffer()
```

- 把一段完整演示存到硬盘
- 清空缓存，准备录下一段

你按**空格**停止录制时执行：

- 把刚才录的 **3 ~ 20 秒视频** 写成一个视频文件
- 把机器人数据保存
- 把这一整段 **Episode** 真正**写入硬盘**
- 然后清空内存缓存，准备录下一段

它 = 保存文件！

### 任务 3：clear → 废弃不要的数据



Python

```
1 self.dataset.clear_episode_buffer()
```

- 录烂了 → 清空缓存
- 不保存，直接丢

#### 任务 4：finalize → 关闭数据集



Python

```
1 self.dataset.finalize()
```

不写数据！只写【目录索引、配置文件】，关闭数据集。

相当于：给文件夹“封口、收尾、关门”。

你按 **ESC 退出程序**时最后执行：

- 写一个 `meta_data.json` （记录总共有多少段视频）
- 写数据集信息
- **关闭所有文件**
- 让数据集变成**可以被加载使用**的状态

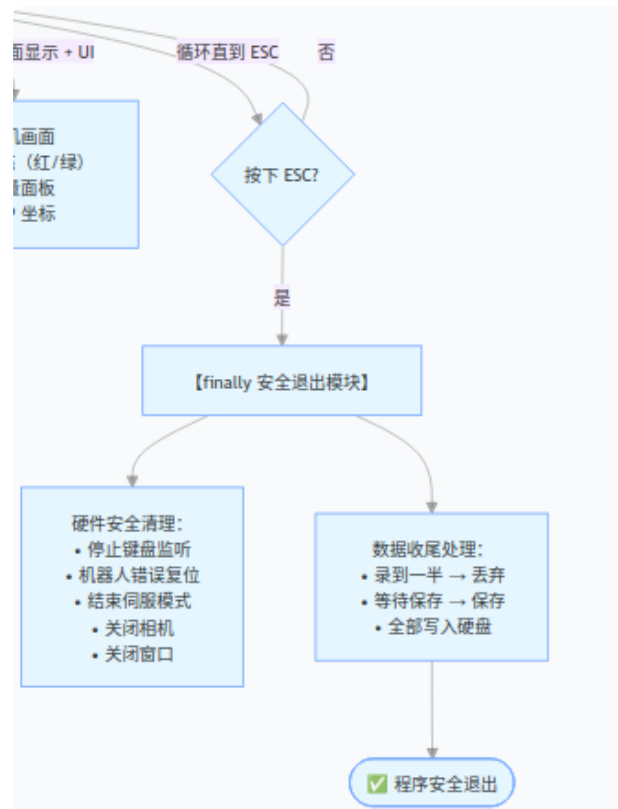
它 = 关门、封口、不写数据！

## 2.怎么实现任务指令的随机抽取的？

就是一个简单的random从列表中随机抽取元素的方法

```
# 空格: 开始/保存录制
if key == keyboard.Key.space:
    if self.record_state == "IDLE":
        # 随机抽取当前任务组的指令
        self.current_episode_prompt = random.choice(TASKS[self.current_group_name]) #从当前任务列表中随机抽取命令
        self.record_state = "RECORDING"
        # 启动质量监控
        self.quality_monitor.start()
        self.idle_frame_count = 0
        print(f"\n[● 录制中] 抽取指令: \"{self.current_episode_prompt}\"")
        print(f"💡 提示: 尽量流畅操作, 减少停顿, 目标15-25秒完成")
    elif self.record_state == "RECORDING":
        self.record_state = "SAVE_PENDING"
```

( 5 ) 在结束程序时各个线程是怎么清理资源的 ?



## 1. finally的作用

首先主循环的run函数的代码框架是：



Python

```
1  try:
2      # 主循环：控制机器人、录制数据
3      while self.running:
4          ...
5  except KeyboardInterrupt:
6      pass
7  finally:
8      # 这里是退出前必须执行的清理代码
9      print("\n正在清理硬件资源...")
10     listener.stop()
11
12     self.robot.ResetAllError()
13     ... # 一大堆关闭、保存、退出逻辑
14
```

**作用：**

**无论程序是正常结束、报错崩溃、还是被强制停止，里面的代码一定、一定、一定会执行。**

**用来做【安全清理、资源释放、数据保存、关机】等绝对不能漏掉的操作。**

**try 负责跑，except 负责抓错，finally 负责“必须善后”。**

**2.清理资源时最后finalize的作用是什么？**

```
# 退出前处理未完成的数据
if self.record_state == "RECORDING":
    print("程序退出，最后半截未完成数据自动丢弃...")
    self.dataset_queue.put({"cmd": "clear"})
elif self.record_state == "SAVE_PENDING":
    self.dataset_queue.put({"cmd": "save"})

# 无论如何，最后都要发送 finalize
# 这个命令的作：把所有缓存数据真正写入硬盘，关闭文件，完成数据集构建
self.dataset_queue.put({"cmd": "finalize"})

print("等待数据后台写入磁盘...")
time.sleep(1.0) #让队列里的 save / clear / finalize 全部执行完
self.running = False
```

### 3.最终版异步推理部署脚本

## 9.离线测试模型推理结果是否合理的脚本（同步推理）

### 1.脚本的作用

▼ Python

```
1  """
2  SmolVLA 模型离线推理测试脚本
3  功能：
4      1. 加载训练好的 SmolVLA 模型
5      2. 打开摄像头获取实时画面
6      3. 使用模拟机械臂状态（不需要真实机器人）
7      4. 让模型预测动作（action）
8      5. 输出：推理速度、动作范围、是否合理
9      6. 检查模型是否可用，是否能用于真实机械臂控制
10 """
11 流程：
12 摄像头画面 → 模型输入
13 假关节状态 → 模型输入
14 任务指令 → 模型输入
15
16 → 模型推理 100 步
17 → 输出 100 个动作
18 → 脚本检查动作是否合理
19 → 告诉你模型是否训练成功
```

## 2.如何使用这个脚本？

打开终端，进入你的 lerobot 环境，运行：

▼ Bash

```
1 python 9_test_inference_offline.py \  
2   --policy-path /home/zjs/Learning_lerobot/lerobot/zjs_test/outputs/fa3_smo  
   lvla_finetune/checkpoints/last/pretrained_model \  
3   --task "put the cube into the box" \  
4   --cam-idx 6
```

### 参数解释（你只需要改这 3 个）

1. `--policy-path` → 你训练好的模型路径，注意！这里必须是全局路径，否则lerobot会认为是hugging face上模型名称，就回去hug上找，找不到就会报错。
2. `--task` → 你想让机械臂做的任务（语言指令）
3. `--cam-idx` → 你的摄像头编号（一般是 0 或 6）

## 3.使用时的注意事项

### （1）传入的训练模型路径

- 如果要加载本地权重，需要是全局路径

### （2）

## 3.对脚本的一些疑问

如果不连机械臂，那模型是怎么推理的？是不是摄像头拍摄到当前的场景，我把摄像头拍摄场景中的机械臂的状态作为当前机械臂状态手动输入给脚本，作为模型的输入之一。然后，模型基于当前摄像头拍摄的图片、我输入的机械臂状态、任务指令，来推理得到一个动作块。而这个推理得到的动作快有多少



步，是不是就取决于--num-steps这个设置的多少。那这个脚本就会分析模型输出的这个动作块是否合理，是这样吗？

## 2.1. 不连机械臂，模型到底怎么推理？

✅ 你理解 完全正确！

推理过程 **完全不需要真实机械臂**，只需要三样东西：

1. 摄像头拍到的当前画面
2. 你手动给的 “假机械臂状态”（代码里写死的初始角度）
3. 任务指令（例如 `put cube into box`）

模型做的事情只有一件：**根据 → 图像 + 关节状态 + 指令输出 → 下一步机械臂应该怎么动**

## 2.2. “机械臂状态” 是哪里来的？

你说得 **完全对**！

```
Python |  
1 fake_state = np.array([-130.0, -98.0, 88.0, -96.0, -90.0, -131.0, 0.0])
```

这就是：

**你手动 “假装” 当前机械臂在这个位置**

模型**完全不知道这是假的**，它只管：

- 看图
- 看状态

- 看指令→ 输出动作

### 2.3. 动作输出多少步，由谁决定？

✅ 完全由 `--num-steps` 决定！

▼

Plain Text

```
1  --num-steps 100
```

意思就是：**让模型连续推理 100 次动作，形成 100 步连续动作**

你理解 **100% 正确**：

- 你设置多少步
- 模型就输出多少步
- 步数 = 推理长度
- 步长 = 10Hz（和你录制时一致）

### 2.4. 这个脚本的作用是不是：分析模型输出的动作是否合理？

✅ 是的！完全正确！

它做的就是：

1. 让模型输出 N 步动作
2. 检查动作是否：
  - 角度不爆炸

- 夹爪在 0~1
- 动作连续不跳变
- 速度够快

3. 最后告诉你：**这个模型能不能上真机运行！**

## 4.脚本的代码

▼ 9\_test\_inference\_offline.py Python |

```
1  # 9_test_inference_offline.py
2  """
3  SmolVLA 模型离线推理测试脚本
4  功能：
5      1. 加载训练好的 SmolVLA 模型
6      2. 打开摄像头获取实时画面
7      3. 使用模拟机械臂状态（不需要真实机器人）
8      4. 让模型预测动作（action）
9      5. 输出：推理速度、动作范围、是否合理
10     6. 检查模型是否可用，是否能用于真实机械臂控制
11  """
12
13  import argparse
14  import time
15  import cv2
16  import numpy as np
17  import torch
18
19  def main():
20      # ===== 1. 解析命令行参数 =====
21      # 从命令行接收：模型路径、任务指令、摄像头编号、设备等
22      parser = argparse.ArgumentParser()
23      parser.add_argument("--policy-path", type=str, required=True, help
24                          ="训练好的模型路径")
25      parser.add_argument("--task", type=str, default="pick up the blue blo
26                          ck and place it into the box")
27      parser.add_argument("--cam-idx", type=int, default=6)
28      parser.add_argument("--device", type=str, default="cuda")
29      parser.add_argument("--num-steps", type=int, default=100, help="测试推
30                          理多少步")
31      args = parser.parse_args()
32
33      # 自动选择设备：GPU 优先
34      device = torch.device(args.device if torch.cuda.is_available() else
```

```
"cpu")
32     print(f"设备: {device}")
33
34     # ===== 2. 加载训练好的 SmolVLA 模型 =====
====
35     print(f"\n[1] 加载模型: {args.policy_path}")
36
37     # 导入 LeRobot 里的 SmolVLA 模型 & 预处理工具
38     from lerobot.policies.smolvla.modeling_smolvla import SmolVLAPolicy
39     from lerobot.policies.factory import make_pre_post_processors
40
41     # 从训练好的 checkpoint 加载模型
42     policy = SmolVLAPolicy.from_pretrained(args.policy_path)
43     policy.to(device)      # 把模型放到 GPU/CPU
44     policy.eval()         # 切换到推理模式 (不训练)
45
46     print(f"    ✓ 模型加载成功")
47     print(f"    输入特征: {list(policy.config.input_features.keys())}")
48     print(f"    输出特征: {list(policy.config.output_features.keys())}")
49     print(f"    Action维度: {policy.config.output_features.get('action',
    {})}")
50
51     # 加载数据预处理/后处理器 (图像归一化、动作缩放等)
52     preprocessor, postprocessor = make_pre_post_processors(
53         policy_cfg=policy.config,
54         pretrained_path=args.policy_path,
55         preprocessor_overrides={"device_processor": {"device": str(device)
    e}}},
56     )
57
58     # ===== 3. 打开摄像头 =====
59     print(f"\n[2] 打开摄像头 index={args.cam_idx}")
60     cap = cv2.VideoCapture(args.cam_idx, cv2.CAP_V4L2)
61     cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)    # 摄像头宽度
62     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)   # 摄像头高度
```

```
62     cap.Set(CV2.CAP_PROP_FRAME_HEIGHT, 480) # 摄像头高度
63
64     if not cap.isOpened():
65         print(" x 摄像头打开失败")
66         return
67     print(" ✓ 摄像头已打开")
68
69     # ===== 4. 构造模拟机械臂状态 =====
70     # 因为不连接机械臂，手动给一个合理的初始关节角度
71     # 格式: [j1, j2, j3, j4, j5, j6, gripper]
72     # 前6个是关节角度，最后1个是夹爪 (0=闭合, 1=张开)
73     fake_state = np.array([-130.0, -98.0, 88.0, -96.0, -90.0, -131.0,
0.0], dtype=np.float32)
74     print(f"\n[3] 模拟初始状态: {fake_state}")
75
76     # ===== 5. 开始推理循环 =====
77     print(f"\n[4] 开始推理测试 ({args.num_steps}步, 任务: \"{args.task}\")")
78     print("-" * 70)
79
80     # 导入工具函数
81     from lerobot.policies.utils import build_inference_frame, make_robot_
action
82     from lerobot.datasets.feature_utils import hw_to_dataset_features
83
84     IMG_SIZE = 256 # SmolVLA 要求图像 256x256
85     STATE_NAMES = ["j1", "j2", "j3", "j4", "j5", "j6", "gripper"]
86
87     # 重置模型、处理器状态 (避免历史信息干扰)
88     policy.reset()
89     preprocessor.reset()
90     postprocessor.reset()
91
92     latencies = [] # 保存每一步推理耗时
93     all_actions = [] # 保存所有输出动作
94
```

```
95     # 循环推理 N 步
96     for step in range(args.num_steps):
97         t0 = time.perf_counter() # 计时开始
98
99         # ----- 读取摄像头画面 -----
100         ret, frame = cap.read()
101         if not ret:
102             continue
103
104         # 缩放到模型需要的 256x256, 并转 RGB
105         frame_resized = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))
106         frame_rgb = cv2.cvtColor(frame_resized, cv2.COLOR_BGR2RGB)
107
108         # ----- 构造模型输入 -----
109         obs_dict = {
110             # 图像: 转成 [1, 3, 256, 256] 格式, 并归一化到 0~1
111             "observation.images.top": torch.from_numpy(frame_rgb).permute
(2, 0, 1).unsqueeze(0).float().to(device) / 255.0,
112             # 模拟机械臂状态
113             "observation.state": torch.from_numpy(fake_state).unsqueeze
(0).float().to(device),
114             # 任务指令 (告诉模型要做什么)
115             "task": args.task,
116         }
117
118         # ----- 数据预处理 (归一化、编码)
119         -----
120         obs_processed = preprocessor(obs_dict)
121
122         # ----- SmolVLA 模型推理 (核心!)
123         -----
124         with torch.no_grad(): # 关闭梯度计算, 加速推理
125             action = policy.select_action(obs_processed)
```



```
125         # ----- 后处理（把模型输出转成真实动作）
126         -----
127         action = postprocessor(action)
128         # ----- 把 action 转成 numpy 数组
129         -----
130         if isinstance(action, dict) and "action" in action:
131             action_np = action["action"].cpu().numpy().flatten()
132         elif isinstance(action, torch.Tensor):
133             action_np = action.cpu().numpy().flatten()
134         else:
135             action_np = np.array(action).flatten()
136         # ----- 计算推理耗时 -----
137         dt_ms = (time.perf_counter() - t0) * 1000
138         latencies.append(dt_ms)
139         all_actions.append(action_np)
140
141         # ----- 打印前10步 + 每20步的动作
142         -----
143         if step < 10 or step % 20 == 0:
144             action_str = ", ".join([f"{v:.2f}" for v in action_np[:7]])
145             print(f" Step {step:3d} | 耗时 {dt_ms:6.1f}ms | action: [{act
ion_str}]")
146         # ----- 用模型输出更新模拟状态（模拟闭环控制）
147         -----
148         if len(action_np) >= 7:
149             fake_state = action_np[:7].astype(np.float32)
150
151         # ----- 画面实时显示 -----
152         display = frame.copy()
153         cv2.putText(display, f"Step {step} | {dt_ms:.0f}ms", (10, 30),
154                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
155         if len(action_nn) >= 7:
```

```
-- .
-- .
155         cv2.putText(display, f"Grip: {action_np[6]:.2f}", (10, 60),
156                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
157     cv2.imshow("Inference Test", display)
158     if cv2.waitKey(1) & 0xFF == 27:
159         break
160
161     # 释放资源
162     cap.release()
163     cv2.destroyAllWindows()
164
165     # ===== 6. 推理结果分析 (最重要!) =====
166     print("\n" + "=" * 70)
167     print("推理结果分析")
168     print("=" * 70)
169
170     # 推理速度统计
171     latencies = np.array(latencies)
172     print(f"推理延迟: 平均={latencies.mean():.1f}ms, 最小={latencies.min():.1f}ms, 最大={latencies.max():.1f}ms")
173     print(f"等效FPS: {1000/latencies.mean():.1f} Hz")
174
175     # 动作范围分析
176     all_actions = np.array(all_actions)
177     print(f"\nAction统计 (共{len(all_actions)}步):")
178     labels = ["J1(°)", "J2(°)", "J3(°)", "J4(°)", "J5(°)", "J6(°)", "Grip
per"]
179     for i, label in enumerate(labels):
180         if i < all_actions.shape[1]:
181             col = all_actions[:, i]
182             print(f" {label:10s}: min={col.min():8.2f}, max={col.max():8.2f}, "
183                   f"mean={col.mean():8.2f}, std={col.std():6.2f}")
184
```

```
185     # ----- 动作合理性检查 -----
186     print("\n合理性检查:")
187     if all_actions.shape[1] >= 7:
188         joints = all_actions[:, :6]    # 前6个是关节
189         gripper = all_actions[:, 6]    # 第7个是夹爪
190
191         # 检查关节角度是否在合理范围 (一般  $\pm 360^\circ$  度)
192         if np.abs(joints).max() < 360:
193             print("    ✓ 关节角度范围合理 (<  $\pm 360^\circ$ )")
194         else:
195             print(f"    ⚠ 关节角度超出范围: max={np.abs(joints).max():.1f}°")
196
197         # 检查夹爪是否在 0~1 附近
198         if gripper.min() >= -0.5 and gripper.max() <= 1.5:
199             print(f"    ✓ 夹爪值范围基本合理: [{gripper.min():.2f}, {gripper.m
200 ax():.2f}]")
201         else:
202             print(f"    ⚠ 夹爪值超出预期: [{gripper.min():.2f}, {gripper.max
203 ():.2f}]")
204
205         # 检查动作是否有变化 (不是完全不动)
206         diffs = np.diff(all_actions, axis=0)
207         max_diff = np.abs(diffs).max()
208         mean_diff = np.abs(diffs).mean()
209         if mean_diff < 0.001:
210             print(f"    ⚠ Action几乎没有变化 (mean_diff={mean_diff:.6f}), 模型
211 可能没学到东西")
212         elif max_diff > 50:
213             print(f"    ⚠ Action跳变过大 (max_diff={max_diff:.1f}), 可能不稳
214 定")
215         else:
216             print(f"    ✓ Action变化幅度合理 (mean_diff={mean_diff:.3f})")
217     else:
218         print(f"    ⚠ Action维度={all_actions.shape[1]}, 预期7维")
219
```

```
215
216     # 速度是否满足实时控制 (≥10Hz 就可以)
217     print("\n✅ 离线推理测试完成!")
218     if latencies.mean() < 100:
219         print(f"推理速度 {1000/latencies.mean():.0f}Hz 满足实时控制要求 (≥10Hz)")
220     else:
221         print(f"推理速度 {1000/latencies.mean():.1f}Hz 偏慢，部署时可能需要降低控制频率")
222
223 if __name__ == "__main__":
224     main()
225
```

## 10.在线部署到法奥3上的推理脚本(同步推理)

## 11.在线部署到法奥3上的推理脚本(异步推理，性能更优)

### (1) 之前脚本的推理方式

当前脚本10的逻辑是每一步都调用一次模型推理，取模型输出的第一个动作执行。这是最简单的同步推理方式：

▼ Python

```
1  循环：
2  1. 读观测 （关节+图像）
3  2. 模型推理 → 得到action
4  3. 执行action (ServoJ)
5  4. 等待下一个周期
6
```

**这种方式的问题是：**

- 模型推理需要时间（比如50-100ms），在推理期间机械臂是"停着等"的，导致运动不连贯。

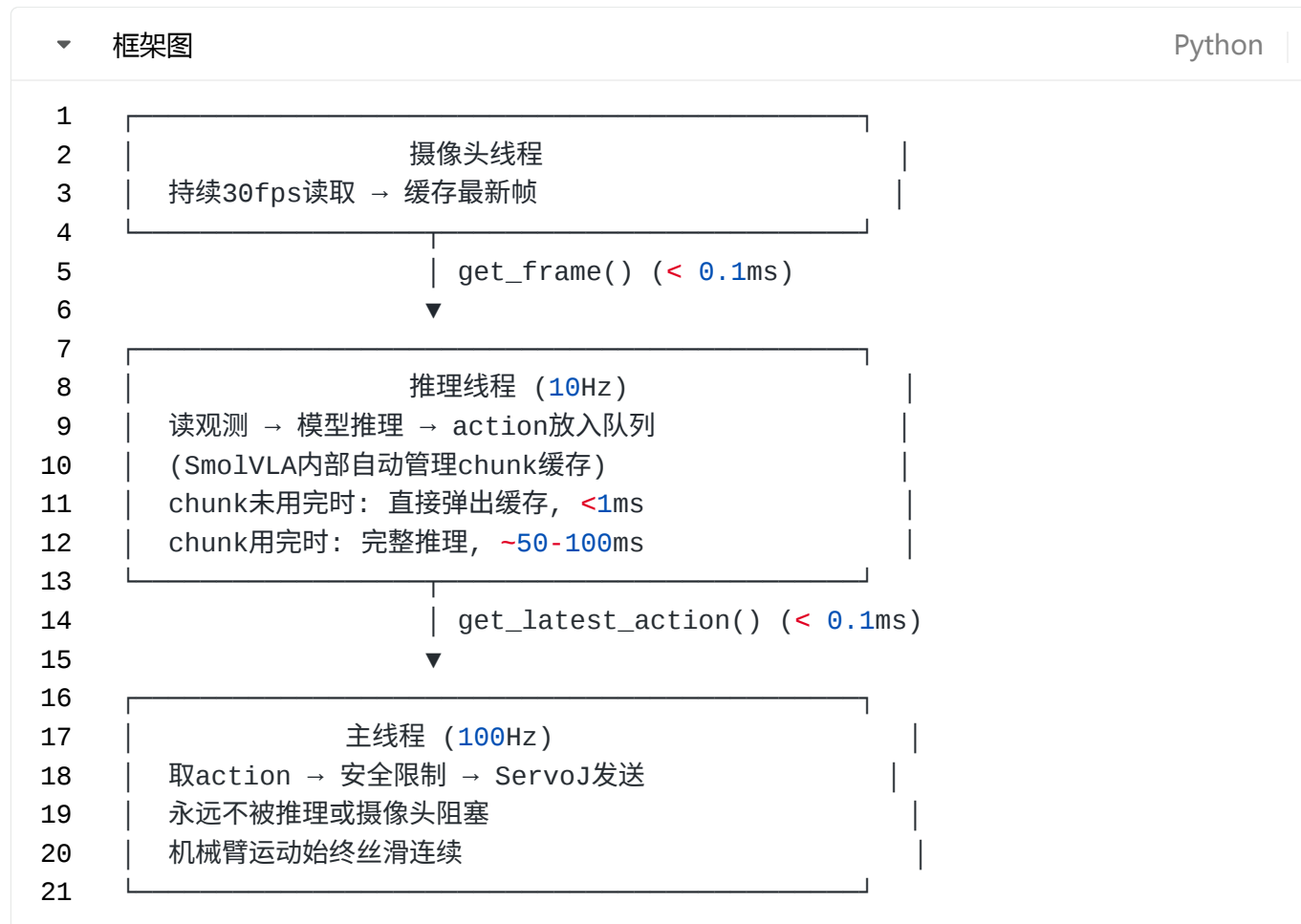
## （2）SmolVLA的动作块(Action Chunk)机制:

- SmolVLA（以及ACT等策略）的核心设计是：模型一次推理不是输出1个动作，而是输出一个动作序列（chunk），比如一次输出未来16步的动作。
- **这个设计的目的是：**
  - 1.模型推理很慢（比如100ms），但机械臂需要高频控制（10-100Hz）
  - 2.模型一次输出16步动作，机械臂可以在模型推理下一个chunk的同时，继续执行之前chunk中剩余的动作
  - 3.这样机械臂的运动就是连续平滑的，不会因为等待推理而"卡顿"
- **具体来说，SmolVLA的select\_action方法内部已经实现了chunk缓存机制：**
  - 1.第一次调用时，模型做完整推理，输出比如16步动作，缓存起来。
  - 2.接下来的15次调用，直接从缓存中弹出下一个动作，不做推理，速度极快
  - 3.第17次调用时，缓存用完，重新做一次完整推理。

- 4.所以你当前脚本10的写法实际上已经在利用chunk机制了，因为policy.select\_action()内部自动管理了chunk缓存。
- **但当前脚本有一个效率问题：它是同步的，即使从缓存取action不需要推理时间，主循环也只运行在10Hz。**

### **( 3 ) 异步推理：终极方案**

- 最优的部署方式是异步推理：模型推理在后台线程运行，主线程以100Hz的固定频率从action队列中取动作发给机械臂。这样无论模型推理多慢，机械臂的控制频率都不受影响。



### 三个线程各司其职：

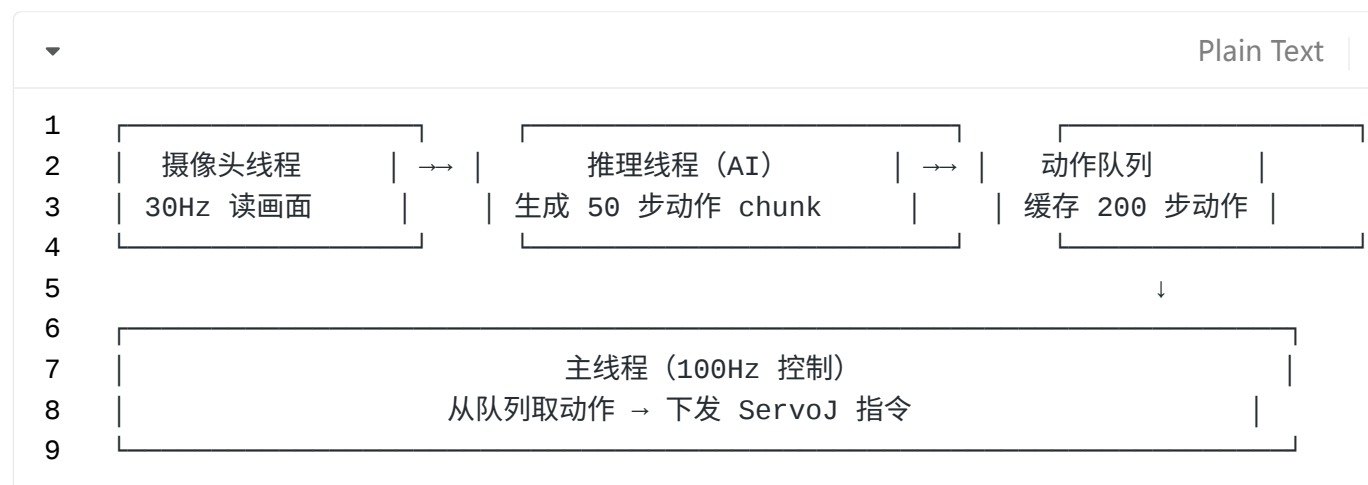
- 1.摄像头线程不停拍照
- 2.推理线程不停思考
- 3.主线程不停控制。

任何一个线程的延迟都不会影响其他线程，这就是为什么机械臂能在模型推理的同时保持100Hz的丝滑控制。

## 12.最终使用版本的异步推理脚本理解

### ( 1 ) 程序架构

三线程+一队列



模块 1 : 摄像头线程 `CameraThread`

作用 : 后台一直读相机 , 不影响控制

- 30Hz 持续读取相机画面
- 只保留**最新一帧**
- 线程安全读取
- 完全独立 , 不阻塞主线程



Python

```
1 def _loop(self):
2     while self.running:
3         ret, frame = self.cap.read()    # 读相机
4         if ret:
5             with self.lock:
6                 self.latest_frame = frame    # 存最新帧
```

## 模块 2：AI 推理线程 `InferenceThread`（最核心）

作用：后台跑模型，生成动作序列，不卡控制

- 队列快空了才推理
- 一次生成 **50 步动作**
- 放入队列给主线程使用
- 夹爪状态用**软件维护**，不从硬件读（解决滞后）

**推理线程执行流程：**

1. 检查队列剩余动作
  - 多于 20 步 → 不推理，睡觉
  - 少于 20 步 → 开始推理
2. 获取相机图像
3. 获取机器人关节角度
4. 获取**软件维护的夹爪状态**（0 = 张开 / 1 = 闭合）

5. 输入模型 → 输出 **50 步连续动作**
6. 放入 `action_queue` 队列

### 模块 3：主线程（主循环 50~100Hz）

**作用：高速、稳定、实时控制机械臂**

- 固定频率运行（50Hz / 100Hz）
- 从队列**取一步动作**
- 动作平滑滤波（让运动更柔顺）
- 下发 `ServoJ` 指令给机械臂
- 处理夹爪开合逻辑
- 显示实时画面

### 完整程序执行流程

#### 【阶段 1：初始化】

1. 加载 SmolVLA 模型
2. 连接法奥机械臂
3. 启动相机后台线程
4. 启动 AI 推理后台线程
5. 机械臂进入伺服模式 `ServoMoveStart()`

#### 【阶段 2：推理线程第一次跑】

- 相机读画面
- 读机器人角度
- 模型推理 50 步动作
- 放入动作队列

### 【阶段 3：主线程开始控制循环】

1. 从动作队列取一步动作
2. 如果队列为空 → **保持当前位置不动**
3. 对关节角度做**平滑滤波**
4. 安全限制：关节每步不能突变太大
5. 判断夹爪是否需要开合
  - 开合 → 暂停伺服 → 动夹爪 → 重启伺服
  - 不合 → 直接发关节指令
6. 下发 `ServoJ` 控制指令
7. 固定频率循环

### 【阶段 4：退出】

- 停止推理线程
- 机械臂退出伺服
- 关闭相机
- 安全关机

**( 2 ) 对于action\_chunk的时间集成机制是怎么做的？有什么效果？为什么叫**

## 时间集成机制？

### 1.实现方式

**时间集成（Temporal Ensembling）**，核心是让**新生成的 action chunk** 与**队列中剩余的旧 chunk** 做**重叠加权融合**，让动作切换更平滑、轨迹更稳定。

#### 1.1 触发时机

推理线程每次生成新 chunk 时，先检查队列剩余动作数：

Python

```
1  with self.lock:
2      old_remaining = len(self.action_queue) # 队列剩余步数
```

- 若 `old_remaining == 0`：队列空，直接把新 chunk 全部放入
- 若 `old_remaining > 0`：队列有旧动作，执行**重叠融合**

#### 1.2 重叠融合核心逻辑（对于重叠部分可以用线性加权平均做平滑，也可以用指数平滑）



Python

```
1  # 取出队列中剩余的旧动作
2  old_actions = list(self.action_queue)
3  self.action_queue.clear()
4
5  # 计算重叠长度（取旧、新chunk中较短的长度）
6  overlap = min(len(old_actions), len(chunk_actions))
7
8  # 对重叠部分做线性加权平均
9  for i in range(overlap):
10     w_new = (i + 1) / (overlap + 1) # 新动作权重递增
11     w_old = 1.0 - w_new             # 旧动作权重递减
12     blended = w_old * np.array(old_actions[i]) + w_new * chunk_actions[i]
13     self.action_queue.append(blended)
14
15  # 新chunk超出重叠的部分直接追加
16  for i in range(overlap, len(chunk_actions)):
17     self.action_queue.append(chunk_actions[i])
```

### 1.3 权重设计（线性插值）

- 重叠区域第  $i$  步：
  - 旧动作权重： $w_{old} = 1 - (i+1)/(overlap+1)$ （随步数递减）
  - 新动作权重： $w_{new} = (i+1)/(overlap+1)$ （随步数递增）
- 效果：越靠近当前，旧 chunk 权重越高，保证过渡到新chunk不突变；越靠后，新chunk 权重越高，实现平滑过渡到新chunk。

### 1.4 权重设计（指数加权平滑）

▼ Python

```
1 self.temporal_weight_decay = 0.01 # k值, 越小越平滑
2
3 def _compute_temporal_weights(self, num_steps):
4     """计算时间集成权重, 越早的步越权重低"""
5     weights = np.exp(-self.temporal_weight_decay * np.arange(num_steps))
6     return weights
```

假设 `num_steps = 5`

▼ Plain Text

```
1 [0, 1, 2, 3, 4]
```

乘以 `self.temporal_weight_decay = 0.01` (k值, 越小越平滑, 因为变化的步长小, 指数下降的就慢, 对应的权重就下降的慢 (从1开始降), 这个权重又是就chunk的权重 (1-这个权重是新chunk的), 也就是说叠加部分, 旧chunk权重下降缓慢, 使得过渡比较平滑, )

▼ Plain Text

```
1 [0, 0.01, 0.02, 0.03, 0.04]
```

加负号: `- 0.01 * i`

▼ Plain Text

```
1 [0, -0.01, -0.02, -0.03, -0.04]
```

送入 `np.exp()` → 指数函数

▼ Plain Text |

```
1  exp(0) = 1
2  exp(-0.01) ≈ 0.99
3  exp(-0.02) ≈ 0.98
4  .....
```

叠加部分的权重状态就是：——平滑的过渡到新的chunk中。

▼ Plain Text

```
1  i=0
2      w_old = 1.00 | █████
3      w_new = 0.00 |
4
5  i=1
6      w_old = 0.90 | █████
7      w_new = 0.10 | █
8
9  i=2
10     w_old = 0.82 | ███
11     w_new = 0.18 | █
12
13  i=3
14     w_old = 0.74 | ██
15     w_new = 0.26 | █
16
17  i=4
18     w_old = 0.67 | ██
19     w_new = 0.33 | ██
20
21  i=5
22     w_old = 0.61 | █
23     w_new = 0.39 | ███
24
25  ...继续往后...
26
27  i=10
28     w_old = 0.37 | █
29     w_new = 0.63 | █████
```



### (3) 如果主线程的消费频率过快，为什么会使得动作执行抖动？或者出现了回退运动的现象？

#### 1. 在什么情况下会出现队列空的情况？和什么有关？

- 我们知道，推理一次的时间大概在200ms左右，每次推理我使用的动作长度是30步，我触发新一次推理的机制是，队列里剩余小于等于10步。
- 初始情况，队列里会有30步（第一次推理），主线程按照频率F开始消费，直到消费到小于等于10时，开始新一次的推理，队列会不会空，关键就看：**剩余这10的消费时间和推理时间哪一个长。**
  - 1. 如果剩余这10步的消费时间（假设500ms）大于推理时间（280ms），那还没走完这剩下的10步，模型就推理出来了新的30步，会和剩下的步数做时间集成，队列又重新变成30步，永远不会空。
  - 2. 但是如果这10步的消费时间（假设200ms）小于推理时间（280ms），主线程消费完了队里剩余的10步后，模型还没有推理出新的30步动作，这个时候，队列就是空的，主线程就会让机械臂**保持当前位置不动**。直到产生了新的30步动作，才继续运行。而且这个时候新的动作是直接加入到队列中给主线程执行，**没办法做时间集成的平滑过渡**，也就是相对于是chunk的**直接拼接执行**，那这样**动作回退**，**抖动**的问题就又会出现了，而且抖动应该会更强烈，因为会有明显的**从停顿到瞬间执行**的现象。
  - 而且每一次chunk之间的衔接都会是这样。
- 所以，队列是否空，与**模型一次产生多少动作步、推理的时机、推理的消耗时间、主线程的执行频率**有关

#### 2. 如果出现了空队列的情况，机械臂在实际执行中会有什么样的效果？

- 队列是空的，主线程就会让机械臂**保持当前位置不动**。直到产生了新的30步动作，才继续运行。而且这个时候新的动作是直接加入到队列中给主线程执行，**没办法做时间集成的平滑过渡**，也就是相对于是chunk的**直接拼接执行**，那这样**动作回退，抖动**的问题就又会出现在，而且抖动应该会更**强烈，因为会有明显的从停顿到瞬间执行的现象**。
- 

#### ( 4 ) 推理线程中模型输入数据类型是什么样的？

Python

```
1  obs_dict = {
2      "observation.images.top": (
3          torch.from_numpy(frame_rgb).permute(2, 0, 1)
4          .unsqueeze(0).float().to(self.device) / 255.0
5      ),
6      "observation.state": (
7          torch.from_numpy(np.array(state_7d, dtype=np.float32))
8          .unsqueeze(0).to(self.device)
9      ),
10     "task": self.task,
11 }
```

一句话总结：

**把相机图像 + 机器人关节状态 + 文本指令，转换成【模型能看懂的 GPU 张量格式】**

- 图像：从 `numpy`数组 → GPU上的float32张量
- 状态：从 `Python`列表 → GPU上的float32张量
- 指令：直接传入字符串

## 1.observation.images.top

Python

```
1  "observation.images.top": (  
2      torch.from_numpy(frame_rgb)  # 1. numpy → torch张量  
3      .permute(2, 0, 1)             # 2. 通道顺序转换 (H,W,C) → (C,H,W)  
4      .unsqueeze(0)                 # 3. 增加 batch 维度  
5      .float()                       # 4. 转成 float32 浮点型  
6      .to(self.device)              # 5. 传到 GPU / CPU  
7      / 255.0                       # 6. 归一化到 [0, 1]  
8  ),
```

### ① `frame_rgb` (推理前)

- 来源：`cv2` 读取 + `resize` + 转 RGB
- 类型：`numpy.ndarray`
- **dtype**：`uint8` (0~255 整数)
- 形状：`(256, 256, 3)`
  - 256 高
  - 256 宽
  - 3 通道 (RGB)

## ② `torch.from_numpy(frame_rgb)`

- **numpy 数组** → **PyTorch 张量**
- 形状： `(256, 256, 3)`
- 类型： `torch.uint8`

## ③ `.permute(2, 0, 1)` 【核心！】

### OpenCV / 图像格式 → PyTorch 模型格式

- 原顺序： `(Height, Width, Channel) → H,W,C`
- 模型要： `(Channel, Height, Width) → C,H,W`

变化： `(256, 256, 3) → (3, 256, 256)`

## ④ `.unsqueeze(0)`

**增加 batch 维度**模型默认接收 **一批数据**，我们现在只输 1 张图，所以加一个 `1` 在最前面。

形状： `(3, 256, 256) → (1, 3, 256, 256)`

## ⑤ `.float()`

**uint8 (0~255 整数) → float32 (浮点数)**神经网络只能算浮点，不能用整数。

类型： `torch.float32`

## ⑥ `.to(self.device)`

把数据传到：

- `cuda:0` ( GPU )
- 或 `cpu`

## ⑦ `/ 255.0`

归一化： $0 \sim 255 \rightarrow 0.0 \sim 1.0$  模型训练时全部做了归一化，推理必须一致。

## 2.observation.state

```
Python
1  "observation.state": (
2      torch.from_numpy(np.array(state_7d, dtype=np.float32)) #np转torch张量
3      .unsqueeze(0) #增加batch维度
4      .to(self.device) #传到 GPU / CPU
5  )
```

## 3.关键的一些转变

- **numpy**  $\rightarrow$  **tensor** : `torch.from_numpy()`
- **HWC**  $\rightarrow$  **CHW** : `permute(2,0,1)` ( 图像必备 )

- 加 batch : `unsqueeze(0)`
- 整数→浮点 + 归一化 : `float() / 255`

#### 4.最终输入模型的数据格式

Python

```
1  obs_dict = {  
2      "observation.images.top": shape=(1, 3, 256, 256) GPU float32,  
3      "observation.state":      shape=(1, 7)          GPU float32,  
4      "task":                   "文本指令"  
5  }
```

### ( 5 ) 为什么推理时一定要加 `policy.eval()` ? 不加会怎么样 ?

`policy.eval()` = 让模型切换到【推理模式】

- 训练时用 `train()`
- 部署运行时用 `eval()`

**必须加 ! 不加模型输出会乱、不准、甚至机械臂发病 !**

#### 1. `eval()` 到底关闭了什么 ?

深度学习模型里有几个只在训练时有用、推理时必须关掉的层 :

1. **Dropout** ( 随机失活 )
  - 训练时 : 随机关掉一些神经元 , 防止过拟合

- 推理时：**必须全部打开**，否则输出随机抖动

## 2. BatchNorm ( 批归一化 )

- 训练时：动态计算均值方差
- 推理时：**固定均值方差**，否则输出不稳定

## 3. 其他训练专用层

`policy.eval()` 做的事：

**把所有 “训练时才开的随机 / 动态层” 全部关掉，固定成稳定模式**

## 2.如果 不加 `policy.eval ()` 会发生什么？

🔥 后果非常严重！

我直接告诉你**真实表现**：

1. **机械臂运动疯狂抖动**Dropout 每次推理输出都不一样
2. **动作完全不准、不可预测**同样画面，每次输出动作都不同
3. **轨迹不连续、乱跳**根本无法完成抓取任务
4. **模型输出和训练时不一致**精度暴跌 30%~80%
5. **夹爪乱开、乱合**完全不可控

总结：

**不加 `eval ()` → 模型处于 “训练抖动模式” → 机械臂直接废了！**

## 3.超简单比喻 ( 秒懂 )

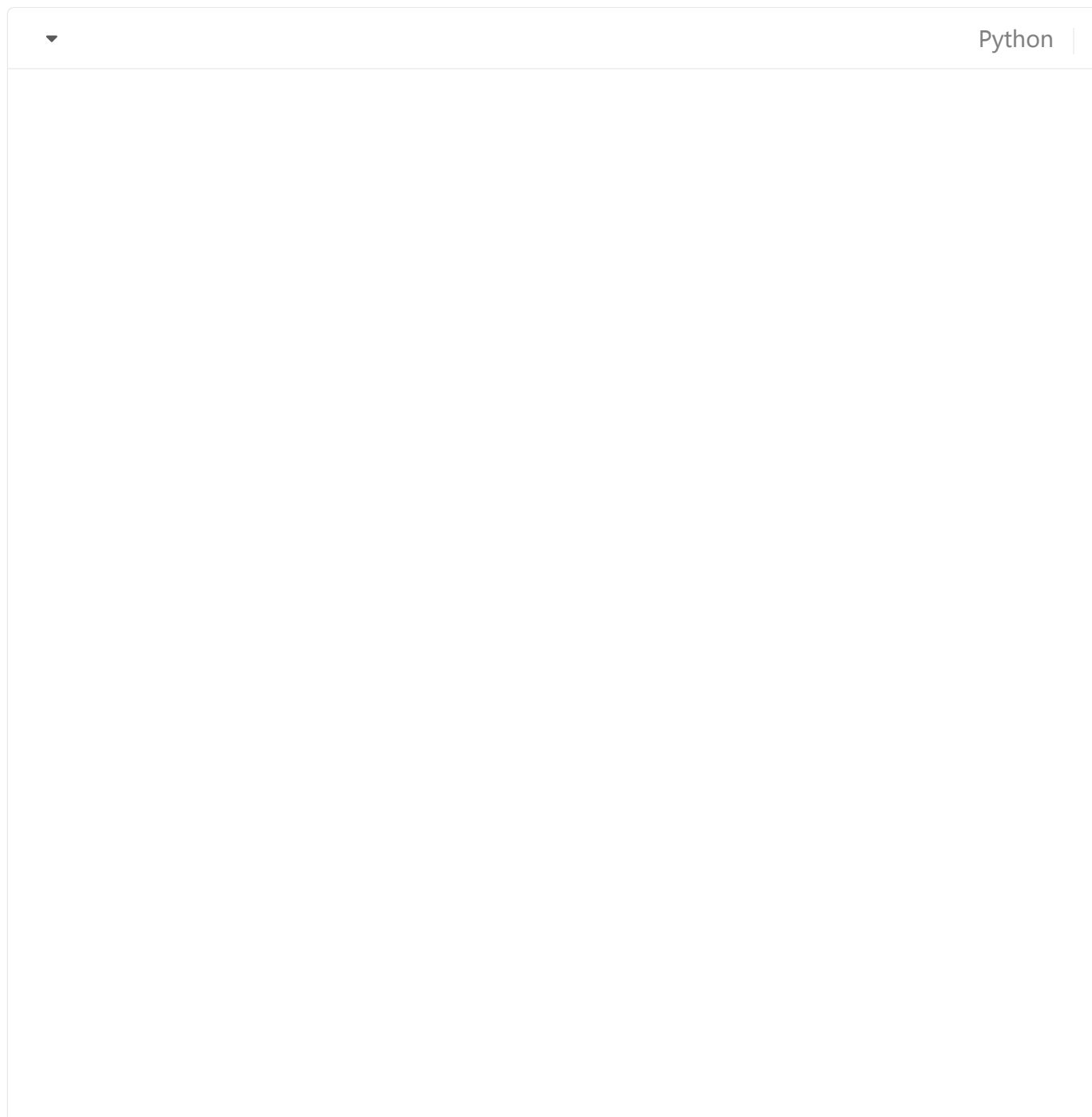
- `train()` 模式 = 学生在练习、尝试、犯错 \*\*
- `eval()` 模式 = 学生在正式考试、稳定发挥 \*\*

机器人部署 = 正式考试必须用 `eval()` ！

## （ 6 ）夹爪的控制逻辑是怎么样的？双阈值滞回，和连续确认机制

- 通过夹爪的当前状态和模型输出的夹爪状态，来判断模型的意图，当连续几次模型的意图都是同一个时，则证明模型真的是想让夹爪的状态变化，而不是输出噪声的跳变。——这就是连续机制确认。
- 而模型输入夹爪的状态就是和双阈值比较的，0.35和0.65。——双阈值带滞回





```
1  grip_value = float(last_action[6]) if len(last_action) >= 7 else 0.0
2  grip_changed = False
3  grip_percent = 0
4
5  GRIP_CLOSE_THRESHOLD = 0.65
6  GRIP_OPEN_THRESHOLD = 0.35
7
8  # 判断意图
9  # current_grip是当前的夹爪状态， 果当前的夹爪实际状态是打开，并且模型输出的夹爪状态预测是大于0.65，也就是模型认为此时夹爪应该闭合
10 # 那就把模型意图状态设置为close
11 if current_grip < 0.5 and grip_value > GRIP_CLOSE_THRESHOLD:
12     intended = "close"
13 elif current_grip >= 0.5 and grip_value < GRIP_OPEN_THRESHOLD:
14     intended = "open"
15 # 果是其他情况，也就是夹爪当前是打开，模型输出也认为该打开，或者模型输出的值在0.35~0.65之间，则认为模型此时没有想改变夹爪状态的意图
16 else:
17     intended = None
18     grip_confirm_counter = 0 #消除模型输出抖动的影响（比 夹爪在关闭状态是，模型预测也是关闭状态，突然模型抖动输出了打开的预测，意图就会变成打开，但是这个噪声跳变，不会持续，所以连续确认机制就发挥作用）
19     grip_pending_state = None
20
21 # 连续确认机制
22 if intended is not None: #说明一定是close或者open的一种
23     # 其实grip_pending_state就是intended的上一次状态
24     if intended == grip_pending_state:
25         grip_confirm_counter += 1
26     else:
27         grip_pending_state = intended
28         grip_confirm_counter = 1
29
30 # 果连续几次，模型的意图都是同一个，则说明模型是真的像让夹爪的状态改变，不是噪声突
```

变

```
31     if grip_confirm_counter >= GRIP_CONFIRM_STEPS:
32         grip_changed = True
33         if intended == "close":
34             grip_percent = args.grip_close_force
35             current_grip = 1.0
36             # 【关键】立即同步给推理线程（新增夹爪状态由软件读取——自己设定的，会有滞后
性——这是用于输入模型的状态）
37             inferencer.set_grip_state(1.0)
38
39             print(f" Step {step:4d} | 夹爪【闭合】 (model_grip={grip_valu
e:.3f}, confirmed)")
40         else:
41             grip_percent = 0
42             current_grip = 0.0
43             # 【关键】立即同步给推理线程
44             inferencer.set_grip_state(0.0)
45             print(f" Step {step:4d} | 夹爪【张开】 (model_grip={grip_valu
e:.3f}, confirmed)")
46             grip_confirm_counter = 0
47             grip_pending_state = None
```

.

## 二、法奥数据采集

### 1.采集过程

#### （1）官方建议：

根据 HuggingFace 的建议，SmolVLA 需要：最少 25 个高质量 episode 才能获得良好性能 推荐 100+ episode 以获得最佳效果 每个 episode 应包含完整的任务执行过程 图像分辨率推荐 224x224 或 256x256

## （ 2 ）采集示例（单任务）

例如：你的任务1：pick up the cube 夹取方块

每个 episode 必须包含完整动作闭环：

1. 机械臂在初始安全位置（我觉的可以设置一个自动回归安全位置的按键）
2. 移动到方块上方
3. 向下垂直接近方块
4. 夹紧夹爪
5. 提起方块
6. 移动到目标位置
7. 放下方块
8. 回到安全位置

每个 episode 时长：8~15 秒

录制频率：10Hz（代码已设置好，不用改）

## （ 2.5 ）模型泛化数据采集

泛化体现的角度：

- 1.物体位置每次都不一样（位置泛化）

- **2.初始姿态每次不一样**（起点泛化）
- **3.少量不同场景 / 光照 / 角度**（视觉泛化）
- **4.自恢复泛化**

## 1. 物体位置泛化（最重要）

**同一个任务，物体每次放得不一样：**

- 方块：左右挪、前后挪、轻微旋转
- 杯子：左右挪、前后挪、杯口朝不同方向
- 盒子 / 杯架：也轻微换位置

**规则：**

- 每次差 **2~5 cm** 即可
- 不要差太大超出工作空间
- 不要每次都放完全同一个点

这样模型才会学会：**“先找到物体 → 再靠近 → 再抓取”** 而不是：**“走到固定坐标就夹”**

## 2. 机械臂起点泛化

每次开始录制前，不要完全回同一个“标准零点”。可以：

- J1 稍微偏一点
- J2 高度稍微高一点 / 低一点
- 整体姿态接近，但**不完全一样**

模型会学会：**不管从哪开始，都能完成任务**而不是：**必须从固定姿势才能执行**

### 3. 视觉泛化（光线、背景、角度）

不用大变，轻微即可：

- 白天 / 傍晚不同光线
- 桌面轻微杂物（不遮挡目标就行）
- 相机角度固定，但允许轻微阴影变化

SmolVLA 是视觉语言模型，**对光照、小遮挡本来就鲁棒**，你只要给一点变化，它就能学到。

### 4. “自然小失误 + 修正”

可以故意录一些：

- 靠近偏了 → 微调再抓
- 没抓稳 → 重新抓一次

模型会学到**更灵活的策略**，而不是僵硬的一条轨迹。

### 具体采集建议

你两个任务可以这样录：

#### 任务 1：put the cube into the box（30 条）

- 前 10 条：方块在**中间区域**
- 中 10 条：方块偏**左**

- 后 10 条：方块偏**右**
- 盒子位置也轻微变化

### 任务 2：put the cup on the cup holder ( 30 条 )

- 10 条：杯子在**近点**
- 10 条：杯子在**远点**
- 10 条：杯子角度不同、杯架位置不同

总数 60 条，泛化性已经很强。

## ( 3 ) 采集步骤 ( 单任务 )

### 步骤 1：准备环境

- 桌面上放一个**方块** ( 颜色明显 )
- 相机固定在上方，能清晰拍到方块
- 机械臂归零，在安全位置

### 步骤 2：启动程序

▼

Bash

```
1 python 7_keyboard_record_fa3.py
```

### 步骤 3：开始录制 ( 每个 episode 都一样 )

① 机械臂回到初始姿势 → 按 空格 开始录制

屏幕出现 **REC 红色**

## ② 控制机械臂完成完整动作

1. 移动到方块上方
2. 下移靠近方块
3. 抓取
4. 提起
5. 放到指定位置
6. 回到安全位置

## ③ 动作完成 → 再按 空格 保存

▼

Plain Text

1



[已保存] Episode 1 成功存入磁盘

## ④ 重复上面流程 25 ~ 100 次

每次录制时可以改变抓取物体的位置，训练模型的空间泛化性

- 最少 25 个 episode
- 推荐 50 ~ 100 个
- 越多效果越稳



## ( 4 ) 采集步骤 ( 多任务 )

### 1.程序修改的地方

打开你的 `7_keyboard_record_fa3.py`

找到这一行：

▼ Python

```
1  def __init__(... task="put the cube into the box"):
```

录任务 A 时：

▼ Python

```
1  task="put the cube into the box"
```

录任务 B 时：

▼ Python

```
1  task="put the cup on the cup holder"
```

每次只改 task 字符串，其他完全不动！

### 2.采集步骤

#### 开始录制任务 A

1. 运行脚本
2. 按空格开始录

3. 执行：**把方块放进盒子**
4. 按空格保存
5. 重复 **25 ~ 50 次**

### 修改 task , 录制任务 B

1. 关闭程序
2. 修改 task 字符串

▼ Python

```
1 task="put the cup on the cup holder"
```

1. 重新运行脚本
2. 录制 把杯子放到杯架
3. 重复 25 ~ 50 次

### 3.最终数据集结构

▼ Plain Text

```
1 local/fa3_smolvla_dataset/  
2 |— episodes/  
3 |   episode_0000 → 放方块  
4 |   episode_0001 → 放方块  
5 |   ...  
6 |   episode_0030 → 放杯子  
7 |   episode_0031 → 放杯子
```

- 每一帧都自带 **task 指令**

- SmolVLA 训练时自动区分
- 推理时你输入什么指令，它就做什么任务

## ( 5 ) 采集必须遵守的 6 条规则

### 1. 每个 episode 必须完整 !

不要中途中断，不要只录一半。

### 2. 动作要慢、平稳

速度控制在 **6 ~ 10 度 / 秒** 最适合训练。

### 3. 方块位置每次稍微变化一点

左右前后挪一点，让模型学会泛化。

### 4. 光线保持一致

不要一会亮一会暗。

### 5. 不要抖动、不要急停

我们的代码是丝滑的，你只需要温柔按键。

### 6. 初始姿势每次尽量一致

让模型学习从固定起点开始执行任务。

## 2.数据集格式 ( lerobot V3.0 )

讲解数据集格式的参考博客:[https://blog.csdn.net/m0\\_63007797/article/details/154176818](https://blog.csdn.net/m0_63007797/article/details/154176818)

```
▼ 标准存储格式 Bash

1  dataset_root/
2  |— meta/                # 元数据目录
3  |   |— info.json        # 数据集schema信息
4  |   |— stats.json       # 全局特征统计 (归一化用)
5  |   |— tasks.parquet    # 任务描述映射
6  |   |— episodes/       # Episode元数据 (分块存储)
7  |       |— chunk-000/
8  |           |— file-000.parquet    # Episode记录 (长度、任务、偏移)
9  |
10 |— data/                # 表格数据目录
11 |   |— chunk-000/
12 |       |— file-000.parquet    # 多个episode的帧数据
13 |
14 |— videos/              # 视频数据目录
15 |   |— front_camera/    # 按摄像头分目录
16 |       |— chunk-000/
17 |           |— file-000.mp4    # 多个episode的视频帧
18 |   |— wrist_camera/
19 |       |— chunk-000/
20 |           |— file-000.mp4
21
```

### (1) chunk-000 到底是什么？

我直接用**最简单、最准确、结合 LeRobot 源码**的方式告诉你：

`chunk-000`、`chunk-001`、`chunk-002` .....

== **数据块 / 分块文件** ==

**就是把大量数据切成一个个小文件包，方便读取、训练、存储。**

## 1. 为什么要分 chunk？（为什么不存成一个大文件？）

因为：

- 如果你录 **1000 个 episode**
- 视频 + 关节数据会**非常大**
- 读一个 **10GB 文件** 很慢
- 训练时**随机读取**也低效

所以 LeRobot（和 HuggingFace 数据集）都采用：

### **分块存储（Chunked Storage）**

- 每 **N 个 episode** 打包成一个 chunk
- 每个 chunk 大小控制在 **几百MB 以内**
- 读取更快、训练更高效、云端传输更友好

## 2. `chunk-000` 到底是什么意思？

- `chunk` = **数据块**

- `000` = **第 0 号数据块**（从 0 开始编号）

当数据越来越多，会自动变成：

```
1 chunk-000
2 chunk-001
3 chunk-002
4 ...
```

你可以把它理解成：

一叠书，chunk 就是每一本书的 “一册”

### 3. 一个 chunk 里包含什么？

LeRobot 标准：

**一个 chunk 包含：多条 episode 的数据**

例如：

- `chunk-000` → episode 0 ~ 20
- `chunk-001` → episode 21 ~ 40
- `chunk-002` → episode 41 ~ 60

### 4. 对应到你的目录结构

```
1 dataset_root/  
2 |— meta/episodes/chunk-000/ → 这段chunk的【元信息】  
3 |— data/chunk-000/ → 这段chunk的【帧数据: state、action、task】  
4 |— videos/xxx/chunk-000/ → 这段chunk的【视频】
```

这三个 chunk-000 是一一对应的！

- 元数据 → 描述这段 chunk
- 数据 → 关节角度、动作
- 视频 → 图像

## 5. 你录少量数据时（比如 10 ~ 30 条 episode）

永远只会出现 chunk-000

因为数据太少，不需要分块。

只有数据量很大时，才会自动生成：

- chunk-000
- chunk-001
- chunk-002
- .....

## 6. 超通俗比喻

- episode = 一张照片

- **chunk = 一本相册**
- **chunk-000 = 第一本相册**

你录的前 20 张照片 → 放进第一本相册 ( chunk-000 )

再录 20 张 → 放进第二本相册 ( chunk-001 )

训练时，模型就是**一本一本相册读取**，而不是看一堆散乱的照片。

### 3.数据集采集具体方案设计



▼ Bash

```
1  任务1（55组）：
2    A. 基础场景（20组）：
3      - 鸭子在工作台前1/3区域随机摆放（包含旋转）
4      - 纸杯在工作台后1/3区域随机摆放
5      - 两者同时随机偏移，每次偏移3~8cm
6      - 操作员从不同高度切入抓取（有时高切入，有时低切入）
7
8    B. 空间大范围泛化（20组）：
9      - 鸭子遍历工作台6个区域（左上/中/右上/左下/中/右下）
10     - 纸杯同样遍历不同区域，且与鸭子不同侧
11     - 每个区域至少3组
12
13   C. 干扰物品（10组）：
14     - 加入盒子、笔等干扰物
15     - 干扰物放在鸭子和纸杯之间，增加难度
16     - 操作员需要绕过干扰物
17
18   # D. 错误恢复（5组）：
19   #   - 故意第一次没抓稳，鸭子掉落
20   #   - 继续操作完成任务
21
22  任务2（55组）： 同上结构
```

## 4.关于 $\pi 0$ 能否直接使用你采集的数据

**可以直接使用，不需要重新采集。**

LeRobot数据集格式是通用格式， $\pi 0$ 、SmolVLA、ACT等主流VLA模型都支持读取同一套数据集。你的数据集已经包含了所有必要的字段：

|   |                        |                          | Bash |
|---|------------------------|--------------------------|------|
| 1 | observation.images.top | → 视觉输入（所有VLA模型都需要）       |      |
| 2 | observation.state      | → 关节状态（所有VLA模型都需要）       |      |
| 3 | action                 | → 动作标签（所有VLA模型都需要）       |      |
| 4 | task                   | → 语言指令（VLA模型需要，BC模型可以忽略） |      |

### 但有以下几点需要注意：

摄像头分辨率：你用的是 $256 \times 256$ ， $\pi 0$ 官方推荐的输入分辨率是 $224 \times 224$ 或者更高。可以在训练时通过数据预处理resize，不需要重新采集。

摄像头数量： $\pi 0$ 原版在训练时用了多个摄像头（腕部+俯视），你只有一个摄像头，泛化效果会比 $\pi 0$ 在多摄像头数据上训练的版本差一些，但不影响基本功能。如果你以后想用 $\pi 0$ ，可以考虑加一个腕部摄像头，但现有数据仍然可以直接用于训练单摄像头版本的 $\pi 0$ 。

关节空间的差异：你的法奥3是标准6DoF工业臂， $\pi 0$ 也支持6DoF+夹爪的7维action空间，完全兼容。

### $\pi 0$ 的训练命令（如果你以后想切换）：

|   |                                                 |  | Bash |
|---|-------------------------------------------------|--|------|
| 1 | python -m lerobot.scripts.train \               |  |      |
| 2 | --policy.type=pi0 \                             |  |      |
| 3 | --policy.pretrained_path=lerobot/pi0_base \     |  |      |
| 4 | --dataset.repo_id=local/fa3_smolvla_multitask \ |  |      |
| 5 | --dataset.video_backend=pyav \                  |  |      |
| 6 | --output_dir=outputs/fa3_pi0_finetune \         |  |      |
| 7 | --batch_size=4 \                                |  |      |
| 8 | --steps=50000                                   |  |      |

数据集路径完全一样，只改policy.type和policy.pretrained\_path即可。

## 三、法奥数据微调训练记录

### 1.云服务器训练环境创建

#### 1.创建文件夹

```
cd data
```

```
mkdir project
```

```
cd project
```

#### 2.创建虚拟环境

```
conda create -n lerobot python=3.12.7 -y
```

#### 3.克隆lerobot仓库

```
git clone https://github.com/huggingface/lerobot.git
```

#### 5.安装lerobot的依赖

```
cd lerobot && pip install -e .
```

## 6.登录wandb

wandb login

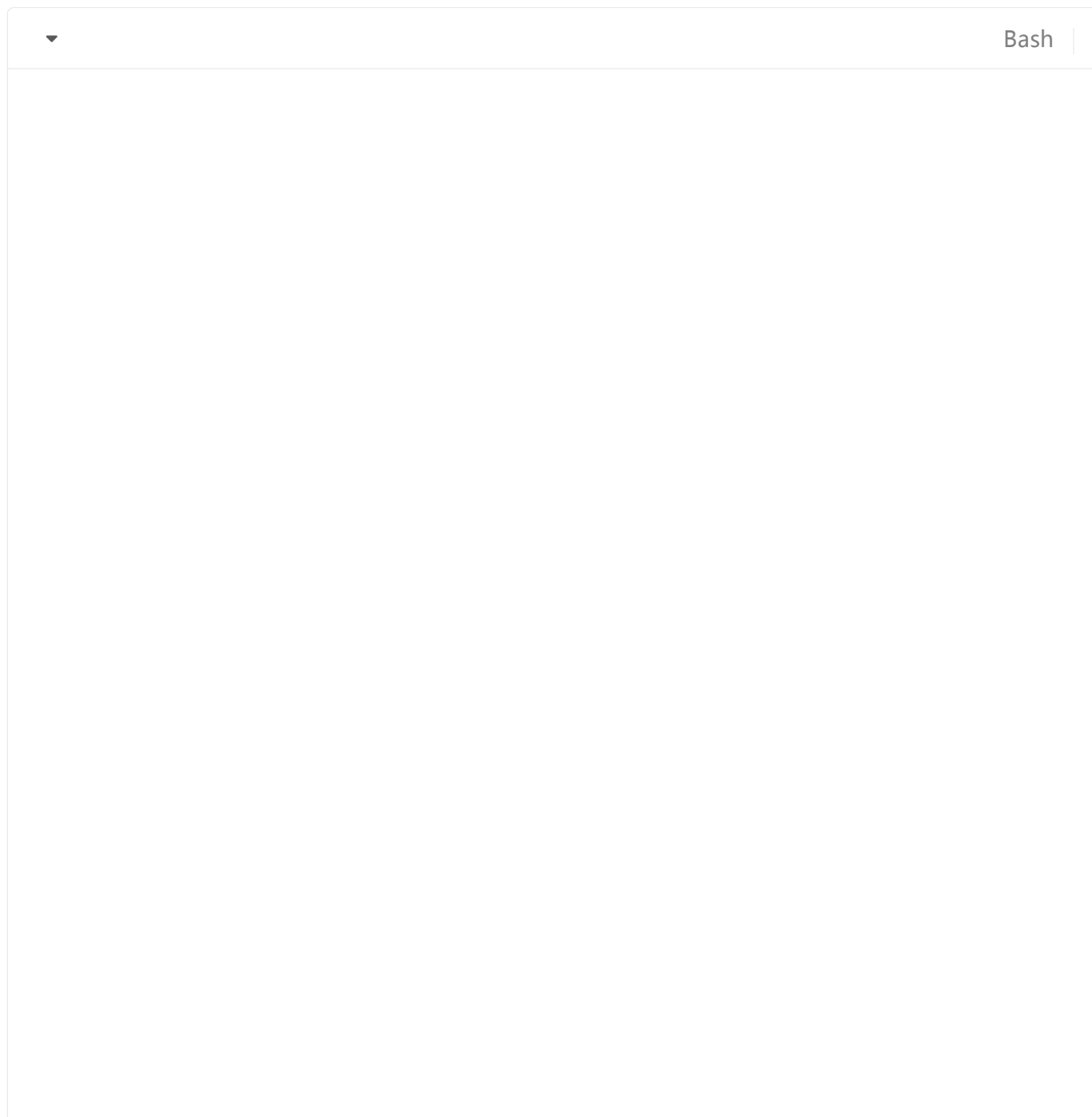
## 7.安装 SmolVLA 依赖项

pip install -e "[smolvla]"

## 4.安装ffmpeg

conda install ffmpeg=7.1.1 -c conda-forge

# 2.微调训练的命令



```
1  最正确的版本：
2  python src/lerobot/scripts/lerobot_train.py \
3      --policy.type=smolvla \
4      --policy.pretrained_path=lerobot/smolvla_base \
5      --dataset.repo_id=/root/data/my_datasets/fa3_smolvla_onetask_final \
6      --output_dir=/root/data/project/outputs/fa3_smolvla_onetask_finetune_fin
al2 \
7      --dataset.video_backend=pyav \
8      --batch_size=32 \
9      --steps=10000 \
10     --save_freq=2000 \
11     --log_freq=50 \
12     --policy.device=cuda \
13     --policy.optimizer_lr=5e-5 \
14     --policy.scheduler_warmup_steps=500 \
15     --policy.scheduler_decay_steps=10000 \
16     --policy.scheduler_decay_lr=1e-6 \
17     --wandb.enable=true \
18     --wandb.project=lerobot_smolvla_fa3_onetask_finetune2 \
19     --wandb.entity="2075817040-" \
20     --policy.push_to_hub=false \
21
22
23  -----
24  python src/lerobot/scripts/lerobot_train.py \
25      --policy.type=smolvla \
26      --policy.pretrained_path=lerobot/smolvla_base \
27      --dataset.repo_id=/root/data/project/datesets/fa3_smolvla_multitask \
28      --output_dir=/root/data/project/outputs/fa3_smolvla_final \
29      --dataset.video_backend=pyav \
30      --batch_size=64 \
31      --steps=20000 \
32      --save_freq=5000 \
33      --log_freq=100 \
```

```
34     --policy.push_to_hub=False
35 # AI给的建议（这个是错误的!!!）
36 python src/lerobot/scripts/lerobot_train.py \
37     --policy.path=lerobot/smolvla_base \
38     --dataset.repo_id=/root/data/my_datasets/fa3_smolvla_onetask_final \
39     --dataset.video_backend=pyav \
40     --output_dir=/root/data/project/outputs/fa3_smolvla_onetask_finetune_fin
al \
41     --batch_size=2 \
42     --steps=10000 \
43     --save_freq=2000 \
44     --log_freq=50 \
45     --policy.device=cuda \
46     --policy.optimizer_lr=5e-5 \
47     --policy.scheduler_warmup_steps=500 \
48     --policy.scheduler_decay_steps=10000 \
49     --policy.scheduler_decay_lr=1e-6 \
50     --wandb.enable=true \
51     --wandb.project=lerobot_smolvla_fa3_onetask_finetune \
52     --wandb.entity="2075817040-"
```

## 1. --policy.type vs --policy.path 的区别

这是两套不同的命令行接口：

| 你的命令 |                                                                   | 官方命令                           |
|------|-------------------------------------------------------------------|--------------------------------|
| 入口   | <code>python src/lerobot/scripts/lerobot_train.py</code>          | <code>lerobot-train</code>     |
| 指定模型 | <code>--policy.type=smolvla + --policy.pretrained_path=...</code> | <code>--policy.path=...</code> |
| 版本   | 旧版/手动脚本方式                                                         | 新版CLI方式                        |

官方的 `--policy.path=lerobot/smolvla_base` 会自动：

- 从路径推断 policy type 是 smolvla
- 加载预训练权重
- 一个参数完成两件事

**你的写法用了两个参数做同一件事，`--policy.type` 告诉框架"创建什么架构"，`--policy.pretrained_path` 告诉框架"从哪里加载权重"。这种写法在旧版API中是必需的，但在新版中被 `--policy.path` 合并了。**

建议：统一用官方新版命令格式。

## 2. `--batch_size=64` — 可能OOM

SmolVLA 是基于视觉语言模型的大架构，24GB 的 4090 跑 `batch_size=64` 大概率显存不足。

## 3. `--steps=20000` — 对50条数据偏多

50条数据，每条大约30-80帧，总帧数约 1500-4000 帧。20000步 × `batch_size` 会把数据重复很多遍，容易过拟合。

## 4. 学习率设置

从你贴的配置文件可以看到SmolVLA的默认学习率设置：



```
▼ Bash
1  # 优化器
2  optimizer_lr: float = 1e-4          # 峰值学习率
3  optimizer_betas: tuple = (0.9, 0.95)
4  optimizer_weight_decay: float = 1e-10
5
6  # 调度器 (不是自动调节, 是预设的schedule)
7  scheduler_warmup_steps: int = 1_000    # 前1000步线性warmup
8  scheduler_decay_steps: int = 30_000    # 30000步余弦衰减
9  scheduler_decay_lr: float = 2.5e-6    # 最终衰减到的最小学习率
```

```
▼ Bash
1  步数:    0  ————— 1000  ————— 30000
2  学习率:  0  → 线性升温 → 1e-4 → 余弦衰减 → 2.5e-6
```

这不是自动调节的，是固定的 Cosine Decay with Warmup 调度策略。

### 3. 默认学习率对你的场景合适吗？

| 因素                                      | 分析                                                     |
|-----------------------------------------|--------------------------------------------------------|
| <code>1e-4</code> 作为微调学习率               | ⚠️ 偏大。通常微调预训练模型用 <code>1e-5</code> ~ <code>5e-5</code> |
| <code>warmup_steps=1000</code>          | ⚠️ 你只训10000步，warmup占了10%，可以接受但偏多                       |
| <code>decay_steps=30000</code>          | ❌ 你只训10000步，衰减根本还没完成就结束了                               |
| <code>freeze_vision_encoder=True</code> | ✅ 冻结视觉编码器，这是正确的微调策略                                    |
| <code>train_expert_only=True</code>     | ✅ 只训练动作专家头，也是合理的                                       |

| 调整说明                   |        |       |                           |
|------------------------|--------|-------|---------------------------|
| 参数                     | 默认值    | 修改值   | 理由                        |
| optimizer_lr           | 1e-4   | 5e-5  | 微调预训练模型应降低学习率，防止灾难性遗忘     |
| scheduler_warmup_steps | 1000   | 500   | 总共10000步，500步warmup占5%更合理 |
| scheduler_decay_steps  | 30000  | 10000 | 与总训练步数对齐，确保学习率完整衰减        |
| scheduler_decay_lr     | 2.5e-6 | 1e-6  | 最终学习率稍微再低一点               |
| eval_freq              | -      | 去掉    | 真实机器人无法在线评估               |

5.LeRobot 训练脚本的命令行参数不支持 `False` 字符串，必须用 `false` 小写！

错误写法：

bash ^

运行

```
--policy.push_to_hub=False
```

正确写法：

bash ^

运行

```
--policy.push_to_hub=false
```

6.数据集的相机名称和smolvla默认相机名称不匹配

你的数据集里的相机叫：`observation.images.top`

但 SmolVLA 模型默认 expecting 3 个相机：

`observation.images.camera1` `observation.images.camera2` `observation.images.camera3`

→ 名字对不上，所以报错！

在你的训练命令里**加入这一行**：

```
▼ Bash
1  --rename_map='{ "observation.images.top": "observation.images.camera1" }'
```

作用：把你的相机 `top` → 映射成模型认识的 `camera1`

- 你录制数据时只用了一个相机：`top`
- 模型默认需要 `camera1`

### 3. 怎么看我的模型训练是否陷入过拟合？

**（1）【看 loss】训练 loss 极低，但不再下降，反而开始上升**

```
▼ Plain Text
1  loss: 0.05 → 0.06 → 0.08 → 0.12 → 0.2
```

**典型过拟合！**

## 为什么这种情况是过拟合？

### 阶段 1：loss 快速下降

模型在学【真正的规律】比如：

- 看到杯子 → 去抓
- 看到盒子 → 放进去
- 夹爪要开合
- 机械臂要移动

这时候模型学得很聪明

### 阶段 2：loss 降到很低，但不再下降

模型已经把“规律”学完了没有新知识可以学了。（可能是由于训练的数据集太小，训练的步数又太多，模型反复的训练这些东西，把能学的规律都学完了）

### 阶段 3：loss 开始上升（关键！）

✅ 这就是过拟合爆发点！

模型发现：

“我学不到更多规律了，那我干脆把训练数据的每一个像素、每一个动作都背下来吧（开始学习数据集的噪声）”

于是开始：

- 记第 1 帧的位置
- 记第 2 帧的角度
- 记第 3 帧的夹爪开度

- 记视频里的噪声
- 记无关的背景变化
- 记录制时的手抖误差

为什么这会让 loss 上升？

这里是最关键的数学逻辑：

模型为了 “完美记住” 某一条数据（泛化能力降低）

会过度调整自己的参数结果导致：

- 记住了这条
- 破坏了之前学的所有其他数据

就像：你为了完美记住第 1 题把第 2~100 题的答案全改错了

→ 整体误差变大 → loss 上升

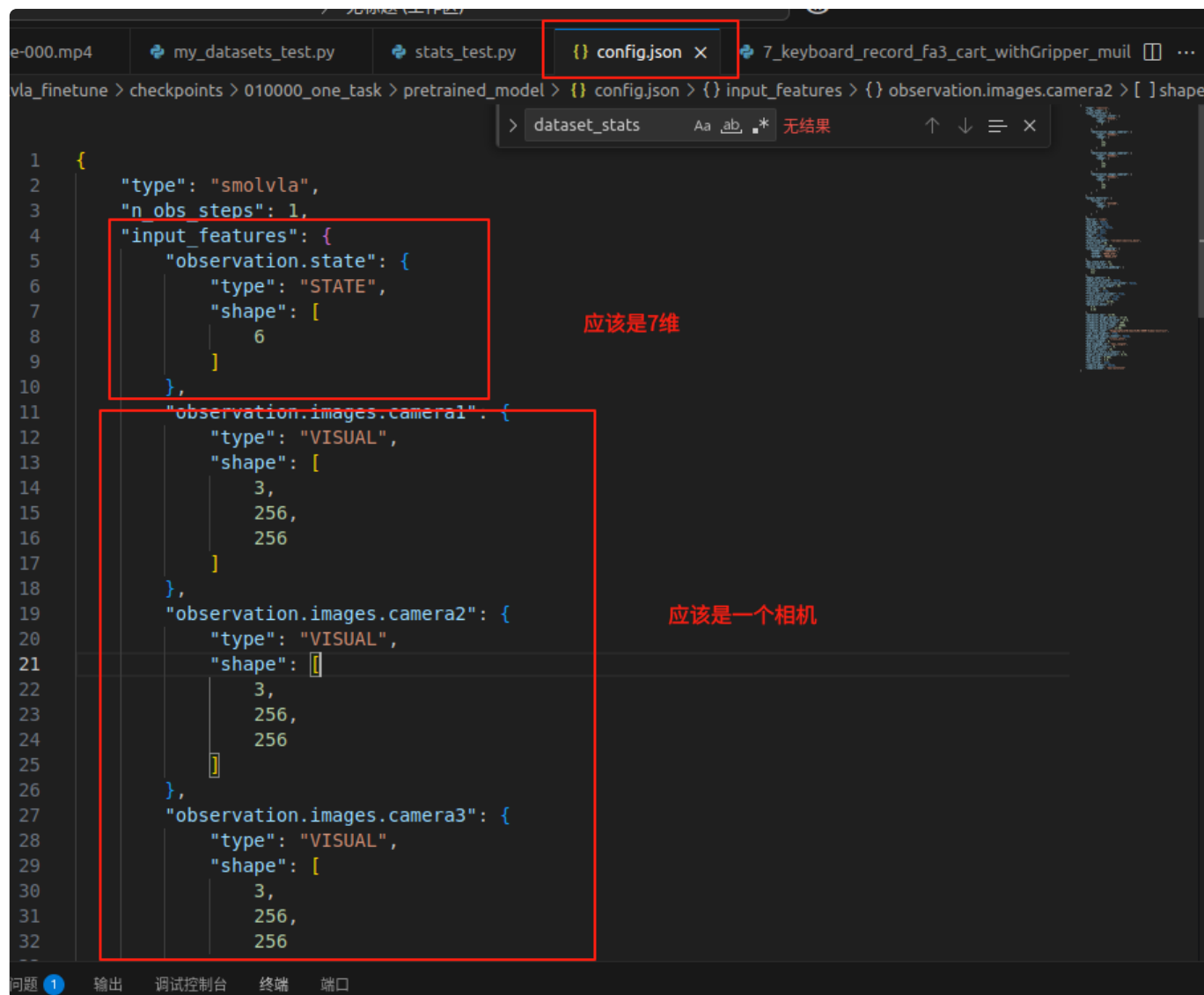
- 先学习真实数据分布（泛化能力↑）
- 再学习训练集噪声（泛化能力↓）
- 噪声越学越多 → 模型参数震荡 → 训练 loss 上升

**（2）【看 loss】训练 loss 非常低（0.01），但推理时机械臂完全不动 / 乱动作**

## 4.微调时预训练模型加载方式的影响！

### （ 1 ）发现问题

我发现我训练出来的模型的配置文件中的输入维度和相机数量和我训练数据不一致！！！



```
1  {
2    "type": "smolvla",
3    "n_obs_steps": 1,
4    "input_features": {
5      "observation.state": {
6        "type": "STATE",
7        "shape": [
8          6
9        ]
10     },
11     "observation.images.camera1": {
12       "type": "VISUAL",
13       "shape": [
14         3,
15         256,
16         256
17       ]
18     },
19     "observation.images.camera2": {
20       "type": "VISUAL",
21       "shape": [
22         3,
23         256,
24         256
25       ]
26     },
27     "observation.images.camera3": {
28       "type": "VISUAL",
29       "shape": [
30         3,
31         256,
32         256
33       ]
34     }
35   }
36 }
```

应该是7维

应该是一个相机

**出现的问题是：**用第二种方式加载预训练模型，预训练的配置会部分覆盖我自己采集的数据集！！也就是我采集的数据集原本输入的state是7维的，相机也只有一个，但是如果采用第二种--  
policy.path=lerobot/smolvla\_base \的加载方式，就会直接加载预训练模型里他自己的配置（3个相机、输入维度是6维）使用第一种加载方式就正常。

## ( 2 ) 问题解决和分析

我第一次测试数据集是否能用于训练使用的指令是：

▼ Bash

```
1  # 测试训练的命令
2  python src/lerobot/scripts/lerobot_train.py \
3      --policy.type=smolvla \
4      --policy.pretrained_path=lerobot/smolvla_base \
5      .....
6
```

我正式训练使用的指令是：

▼ Bash

```
1  python src/lerobot/scripts/lerobot_train.py \
2      --policy.path=lerobot/smolvla_base \
3      .....
4      --rename_map='{ "observation.images.top": "observation.images.camera1" }'
```

他们最主要的问题就是，在加载lerobot/smolvla\_base预训练权重时的方式不同。



这个问题和你的数据、你的参数、你的训练过程没有任何关系，这纯粹是LeRobot框架两种不同的模型加载API有完全不同的隐藏行为，并且官方文档完全没有提到过这个区别，甚至还主动推荐有问题的那个格式。

## 两种加载API的核心行为对比

我把框架内部的执行顺序拆解给你看：

| 命令格式                                                                                                                                            | 内部执行顺序                              | State维度 | 多余相机    | 整体正确性                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|---------|---------|------------------------------------------------------------------------------------------------|
|  <code>--policy.type=smolvla + --policy.pretrained_path</code> | <div>先创建空白配置，从数据集自动检测维度，再加载权重</div> | 7维 完全正确 | 0个      |  所有维度全部正常工作 |
|  <code>--policy.path=lerobot/smolvla_base</code>               | <div>先完整加载预训练模型里硬编码的旧配置，再部分覆盖</div> | 6维 被截断  | 2个多余空相机 |  夹爪输入永久丢失   |

### 1. 第一种正确的加载模型的方式内部的工作过程

▼

Bash

```
1  --policy.type=smolvla \  
2  --policy.pretrained_path=lerobot/smolvla_base \  

```

框架内部执行顺序：

1. 创建一个全新的空白的 `SmolVLAConfig` 对象
2. 扫描你的数据集，自动检测所有feature的正确形状

3. 用检测到的形状完整覆盖config里所有的shape字段
4. 现在config已经100%适配你的数据集了
5. 最后一步：加载预训练权重，匹配到已经调整好的模型结构

这也就是为什么你第一次跑的旧命令输出的config是完美的：

- 自动识别到7维state
- 自动识别到只有一个相机 observation.images.top
- 不需要任何手动的rename map
- 没有任何多余的空相机槽位

这个格式是目前社区里所有成功部署单臂SmolVLA的人实际使用的格式。

## 2. 第二种错误的模型加载方式

▼

Bash

```
1 python src/lerobot/scripts/lerobot_train.py \  
2   --policy.path=lerobot/smolvla_base \
```

框架内部执行顺序：

1. 直接从Hugging Face完整下载 lerobot/smolvla\_base
2. 不加修改的加载这个模型里面保存的旧的硬编码配置：
  - 硬编码 observation.state.shape = [6]
  - 硬编码3个相机 camera1 / camera2 / camera3
1. 然后才扫描你的数据集
2. **✗ 框架只会覆盖输出维度，永远不会覆盖输入维度！**
3. 然后要求你提供手动的rename map把你的相机名映射到旧config里的名字

这个就完美解释了你看到的对称的bug：

| 组件       | 结果     | 原因                |
|----------|--------|-------------------|
| Action输出 | ✅ 7维正确 | 输出维度会被数据集覆盖       |
| State输入  | ❌ 永久6维 | 输入维度永远保留预训练模型里的旧值 |

## 四、真机部署遇到的一些问题

### 1.什么是伺服关节运动？servoJ和moveJ的区别？

#### 1.1什么是 ServoJ 伺服关节运动？

用最简单的话讲：

✅ ServoJ = 实时、高频、连续的关节伺服控制

- 每隔几毫秒（8ms）发一次目标角度
- 机器人立刻跟踪这个目标
- 像手控操纵杆一样，实时响应
- 专门用于 AI 实时控制 / 机器人示教 / 部署运行

#### 1.2 ServoJ（伺服运动）vs 普通点到点运动（P2P）

## 1 ) 普通点到点运动 ( MoveJ / 点动 JOG )

- 给一个目标点
- 机器人自己规划路径、速度、加速度
- 发一次指令，机器人自己跑完
- 特点：粗、慢、不实时
- 用途：搬运、上下料、固定动作

## 2 ) ServoJ 实时伺服运动

- 每隔 8ms 发一个新目标
- 机器人立刻跟踪，不做路径规划
- 连续、平滑、实时响应
- 特点：细、快、极实时
- 用途：AI 控制、手动示教、远程操作、LeRobot 部署

## 1.3 最直观的比喻

### 普通点到点运动 ( MoveJ )

= 你告诉车：开到 100 米外，车自己开过去

### ServoJ 实时伺服

= 你手里拿着方向盘，每秒转 125 次方向，车立刻跟着转

## 1.4 为什么 LeRobot 必须用 ServoJ ?

因为：

- **AI 模型每 8~10ms 输出一个新动作**
- 机器人必须**立刻执行**
- 不能等 “点到点运动慢慢跑完”
- 必须用 **ServoJ 实时跟踪**

## 2.抖动！直接使用sveroJ进行键盘控制机械臂非常抖动！和我在moviet混合规划真机部署时遇到的问题一模一样（这次可以解决啦！！）

### （1）为什么直接使用servoJ会出现异常抖动的现象？

- **抖动现象**：噪音很大，转一个关节，连带着其他关节也跟着抖动
- **原因分析**：
  - 在之前的代码中，控制逻辑是：“读取当前**实际位置** -> 加上步长 -> 发送给机器人”。因为物理机械臂在运动时**实际位置**会有微小的跟随误差和传感器读取噪声。当你不断把**带有微小噪声**的实际位置作为下一个周期的**目标位置**发送给 ServoJ 时，**噪声就被放大了**（所谓的“**闭环正反馈震荡**”），这也就是为什么你感觉“没按的关节也在跟着抖抖抖（因为在读取的时候其他传感器有噪声，所以在读取其他关节时，读取的实际是加了噪声的关节位置，在把这个发给servoJ，执行时其他关节肯定会抖动）”。而在第 4 个脚本中，正弦波是纯数学生成的，没有叠加传感器的读取噪声，所以极

其丝滑。

• 旧版抖动原因：

- a. 每一步都**读取真实关节位置**（带噪声、误差）
- b. 用这个**带噪声的值**当基础去加步长
- c. 噪声被不断放大 → **正反馈震荡** → **抖动、漂移、其他轴乱晃**

## （2）如何解决？

新版解决思路：

**彻底不使用实时读取的关节值来计算目标！**

→ 内部维护一个**纯数学、无噪声、干净**的目标角度

→ 按键只修改这个**干净目标**

→ 直接发给机器人

→ **完全切断噪声反馈环路**

**初始化 只读取一次真实位置**

```
▼ Python |
1  # 【关键】初始化时，只读取一次当前位置作为基准
2  err, current = self.robot.GetActualJointPosDegree()
3  self.target_joints = list(current)
```

作用：

**只在开始对齐一次之后永远不再用真实位置计算目标**→ **彻底切断噪声环路**

## 运动计算 完全基于纯数学

Python

```
1  # 根据按键，纯数学计算下一个目标位置
2  for k in keys:
3      if k in self.KEY_MAP:
4          joint_idx, direction = self.KEY_MAP[k]
5          # 每次循环移动的角度 = 速度 * 周期时间
6          self.target_joints[joint_idx] += direction * self.speed_deg_per_sec
          * dt
```

目标角度 完全由公式算出和机器人当前实际反馈无关→ 没有噪声、没有误差、没有漂移

就像你之前的正弦摆动一样丝滑！

## 不再读取实时关节用于控制

旧版：

Python

```
1  err, current = robot.GetActualJointPosDegree() # 读噪声
2  target = current + step                       # 噪声叠加
```

新版：

Python

```
1  # 【完全没有】读取真实关节来计算目标！
```

机器人反馈只用于观测，不用于控制！→ 正反馈震荡回路被彻底切断

### （3）这种软件开环控制会不会导致误差累积？

在你的 LeRobot 数据集采集、键盘控制场景里：

完全没问题！不仅没问题，而且是【最优方案】！

#### 软件开环位置控制 + 机器人底层硬件闭环

结构是这样的：

1. 电脑（你代码里）

开环：只发目标，不读反馈来修正目标

2. 机器人控制器（内部）

硬件闭环：它自己会疯狂跟踪目标，误差  $< 0.01^\circ$

**这种硬件闭环的精度很高，所以几乎每次发给他的目标值他都能准确达到，因此也就不会存在我们所担心的累积误差。相当于我给他发理想位置，他也几乎能理想到达（这是硬件闭环的功劳！）所以我们不需要自己在软件上再闭环了**

**LeRobot 所有官方控制方案，全都是这种“开环发目标”方式！**

包括：

- 示教采集
- 远程操作
- AI 推理部署
- 所有伺服控制 (ServoJ)

**行业标准做法：**



电脑只管发目标，机器人自己负责跟踪！

### 3. 夹爪的运动超时问题

- 为什么在控制夹爪时，示教器会报运动超时？
- 这是因为我们在设置夹爪控制指令时，设置的夹爪最大运动等待时间太小了，夹爪从开到闭合所需要的实际时间大于我们设置的最大等待时间，这就会导致运动超时。
- 把时间设置得大一些就可以了

### 4. 录制数据集时，机械臂控制卡顿问题

当我开启录制时，在用键盘控制机械臂，机械臂运动就比较卡顿，如果不开启录制，键盘控制机械臂运动就比较丝滑，这是什么原因？怎么解决？

#### 卡顿的原因是：

录制时每隔 10 个控制循环（100Hz/10Hz=每10个循环）会执行一次 `cap.read()` + `cv2.cvtColor()` + `cv2.imshow()` + `cv2.waitKey()`，这些操作加起来大约需要 40ms（你之前测试过摄像头读取延迟是 28ms），而 100Hz 控制循环每次只有 10ms 的时间窗口。**\*\*这意味着每次录制帧时，会连续错过 34 个 ServoJ 指令的发送时机\*\***，控制器收到的指令流就出现了“断档”，机械臂自然就卡顿。

#### 解决方法：

把摄像头读取和图像处理放到独立线程中，主线程只负责 100Hz 的 ServoJ 控制，永远不被阻塞。

- **1.新增 CameraThread 类：**一个独立线程持续以 30fps 读取摄像头，把最新帧缓存在内存中。主线程需要图像时直接调用 `self.camera.get_frame()`，返回的是内存中已缓存好的帧拷贝，耗时不到 0.1ms，完全不阻塞 ServoJ 的 100Hz 控制循环。
- **2.图像缩放到 SmolVLA 推荐分辨率：**在录制帧时执行 `cv2.resize(frame, (256, 256))`，数据集 features 中 shape 也改为 (256, 256, 3)。你可以在脚本顶部将 `IMG_SIZE = 256` 改为 224。
- **3.画面预览降频到 10fps：**用时间戳控制预览刷新闻隔为 0.1 秒，且预览帧也来自缓存而非重新读取，彻底消除了 `cv2.imshow` 对控制循环的影响。

▼ 修改后的采集脚本

Bash

```
1  # 7_keyboard_record_fa3.py
2  import sys
3  import time
4  import cv2
5  import threading
6  import numpy as np
7
8  try:
9      from pynput import keyboard
10 except ImportError:
11     print("请安装: pip install pynput")
12     sys.exit(1)
13
14 from lerobot.datasets.lerobot_dataset import LeRobotDataset
15 from fairino.Robot import RPC
16
17 TASK_LIST = [
18     "pick up the blue block and place it into the box",
19     "pick up the red cup and hang it on the cup rack",
20 ]
21
22 IMG_SIZE = 256 # SmolVLA推荐 224 或 256
23
24
25 class CameraThread:
26     """独立线程持续读取摄像头，主线程随时可以取最新帧，零延迟。"""
27     def __init__(self, cam_idx, width=640, height=480):
28         self.cap = cv2.VideoCapture(cam_idx, cv2.CAP_V4L2)
29         self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
30         self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
31         self.cap.set(cv2.CAP_PROP_FPS, 30)
32         self.latest_frame = None
33         self.lock = threading.Lock()
34         self.running = True
```

```
35         self.thread = threading.Thread(target=self._loop, daemon=True)
36         self.thread.start()
37
38     def _loop(self):
39         while self.running:
40             ret, frame = self.cap.read()
41             if ret:
42                 with self.lock:
43                     self.latest_frame = frame
44
45     def get_frame(self):
46         with self.lock:
47             return self.latest_frame.copy() if self.latest_frame is not None
48         else None
49
50     def stop(self):
51         self.running = False
52         self.thread.join(timeout=2)
53         self.cap.release()
54
55     class FA3MultiTaskRecorder:
56         KEY_MAP = {
57             '1': (0, +1), 'q': (0, -1),
58             '2': (1, +1), 'w': (1, -1),
59             '3': (2, +1), 'e': (2, -1),
60             '4': (3, +1), 'r': (3, -1),
61             '5': (4, +1), 't': (4, -1),
62             '6': (5, +1), 'y': (5, -1),
63         }
64         STATE_NAMES = ["j1", "j2", "j3", "j4", "j5", "j6", "gripper"]
65
66     def __init__(self, ip="192.168.58.2", cam_idx=2,
67                 repo_id="local/fa3_smolvla_multitask"):
```

```
68         self.ip = ip
69         self.cam_idx = cam_idx
70         self.repo_id = repo_id
71         self.current_task_idx = 0
72         self.current_task = TASK_LIST[0]
73         self.control_hz = 100
74         self.record_fps = 10
75         self.speed_deg_per_sec = 10.0
76         self.target_joints = [0.0] * 6
77         self.gripper_command = None
78         self.gripper_target_normalized = 0.0
79         self.running = True
80         self.record_state = "IDLE"
81         self.pressed_keys = set()
82         self.lock = threading.Lock()
83         self.task_episode_counts = {t: 0 for t in TASK_LIST}
84
85     def connect(self):
86         print(f"连接机器人 {self.ip}...")
87         self.robot = RPC(self.ip)
88         time.sleep(1.5)
89         self.robot.ResetAllError()
90         self.robot.RobotEnable(1)
91         time.sleep(0.3)
92         self.robot.Mode(0)
93         time.sleep(0.3)
94
95         err, current = self.robot.GetActualJointPosDegree()
96         if err == 0:
97             self.target_joints = list(current)
98
99         self.gripper_target_normalized = float(self.robot.robot_state_pk
100 g.gripper_position) / 100.0
101         self.robot.ServoMoveStart()
```

```
102         # 独立线程读取摄像头
103         self.camera = CameraThread(self.cam_idx)
104
105         features = {
106             "observation.images.top": {
107                 "dtype": "video",
108                 "shape": (IMG_SIZE, IMG_SIZE, 3),
109                 "names": ["height", "width", "channel"],
110             },
111             "observation.state": {
112                 "dtype": "float32",
113                 "shape": (7, ),
114                 "names": self.STATE_NAMES,
115             },
116             "action": {
117                 "dtype": "float32",
118                 "shape": (7, ),
119                 "names": self.STATE_NAMES,
120             },
121         }
122         self.dataset = LeRobotDataset.create(
123             repo_id=self.repo_id,
124             fps=self.record_fps,
125             robot_type="fa3",
126             features=features,
127             use_videos=True,
128         )
129         self.dataset.clear_episode_buffer()
130
131     def _switch_task(self):
132         if self.record_state == "RECORDING":
133             print("[警告] 录制中不能切换任务")
134             return
135         self.current_task_idx = (self.current_task_idx + 1) % len(TASK_LI
```

---

```
ST)
136         self.current_task = TASK_LIST[self.current_task_idx]
137         count = self.task_episode_counts[self.current_task]
138         print(f"\n📋 切换任务 [{self.current_task_idx+1}/{len(TASK_LIS
T)}}]")
139         print(f"    \"{self.current_task}\" (已录 {count} 条)")
140
141     def _on_press(self, key):
142         try:
143             k = key.char
144         except:
145             k = None
146         if k:
147             with self.lock:
148                 self.pressed_keys.add(k)
149                 if k == '=' or k == '+':
150                     self.speed_deg_per_sec = min(self.speed_deg_per_sec + 2.0
, 30.0)
151                 elif k == '-':
152                     self.speed_deg_per_sec = max(self.speed_deg_per_sec - 2.0
, 2.0)
153                 elif k == 'o':
154                     self.gripper_command = "open"
155                 elif k == 'c':
156                     self.gripper_command = "close"
157             if key == keyboard.Key.tab:
158                 self._switch_task()
159             if key == keyboard.Key.space:
160                 if self.record_state == "IDLE":
161                     self.record_state = "RECORDING"
162                     print(f"\n🔴 录制] \"{self.current_task}\"")
163                 elif self.record_state == "RECORDING":
164                     self.record_state = "SAVE_PENDING"
165                     print("\n🕒 保存中]...")
166             if key == keyboard.Key.esc:
```



```

167         self.running = False
168
169     def _on_release(self, key):
170         try:
171             k = key.char
172         except:
173             k = None
174         if k:
175             with self.lock:
176                 self.pressed_keys.discard(k)
177
178     def _get_state_7d(self):
179         err, joints = self.robot.GetActualJointPosDegree()
180         if err != 0:
181             return None
182         grip_norm = float(self.robot.robot_state_pkg.gripper_position) /
183         100.0
184         return list(joints) + [grip_norm]
185
186     def _get_action_7d(self):
187         return list(self.target_joints) + [self.gripper_target_normalize
188         d]
189
190     def _print_summary(self):
191         print("\n📊 各任务统计:")
192         total = 0
193         for i, t in enumerate(TASK_LIST):
194             c = self.task_episode_counts[t]
195             total += c
196             m = "👉" if i == self.current_task_idx else ""
197             print(f" [{i+1}] {c:3d} 条 | \"{t}\"{m}")
198         print(f" 总计: {total} 条")
199
200     def run(self):

```

```
199         self.connect()
200         listener = keyboard.Listener(on_press=self._on_press, on_release=
self._on_release)
201         listener.start()
202
203         print("\n" + "=" * 60)
204         print(f"🐼 多任务采集 | 图像: {IMG_SIZE}x{IMG_SIZE} | 7维action")
205         print("关节: 1/q ~ 6/y  夹爪: o/c  速度: +/-")
206         print("SPACE: 录制/保存  TAB: 切换任务  ESC: 退出")
207         print(f"数据路径: {self.dataset.root}")
208         for i, t in enumerate(TASK_LIST):
209             m = "👉" if i == 0 else ""
210             print(f" [{i+1}] \'{t}\' {m}")
211         print("=" * 60)
212
213         dt_control = 1.0 / self.control_hz
214         record_interval = int(self.control_hz / self.record_fps)
215         loop_counter = 0
216         gripper_cooldown = 0
217         last_display_time = 0
218
219         try:
220             while self.running:
221                 t0 = time.perf_counter()
222
223                 # --- 1. 保存 (主线程) ---
224                 if self.record_state == "SAVE_PENDING":
225                     self.dataset.save_episode()
226                     self.dataset.clear_episode_buffer()
227                     self.task_episode_counts[self.current_task] += 1
228                     print(f"✅ 已保存! 总: {self.dataset.num_episodes}")
229                     self._print_summary()
230                     self.record_state = "IDLE"
231
232                 # --- 2. 夹爪 (主线程串行) ---
```

```
232 # --- 2. 夹爪 (非线性控制) ---
233 if self.gripper_command and gripper_cooldown <= 0:
234     self.robot.ServoMoveEnd()
235     time.sleep(0.02)
236
237     if self.gripper_command == "open":
238         self.robot.MoveGripper(1, 100, 50, 50, 30000, 0,
0, 0, 0, 0)
239         self.gripper_target_normalized = 1.0
240         print("夹爪: 张开")
241     elif self.gripper_command == "close":
242         self.robot.MoveGripper(1, 0, 50, 50, 30000, 0, 0,
0, 0, 0)
243         self.gripper_target_normalized = 0.0
244         print("夹爪: 闭合")
245
246     self.gripper_command = None
247     gripper_cooldown = 50
248     time.sleep(1.0)
249     self.robot.ResetAllError()
250     time.sleep(0.1)
251
252     err, current = self.robot.GetActualJointPosDegree()
253     if err == 0:
254         self.target_joints = list(current)
255         self.robot.ServoMoveStart()
256         time.sleep(0.02)
257
258 if gripper_cooldown > 0:
259     gripper_cooldown -= 1
260
261 # --- 3. 100Hz 关节控制 (永远不被阻塞) ---
262 with self.lock:
263     keys = set(self.pressed_keys)
264     for k in keys:
```

```
265             if k in self.KEY_MAP:
266                 idx, d = self.KEY_MAP[k]
267                 self.target_joints[idx] += d * self.speed_deg_per
                _sec * dt_control
268
269             self.robot.ServoJ(self.target_joints, [0, 0, 0, 0],
270                               acc=0.0, vel=0.0, cmdT=dt_control, filt
                erT=0.0)
271
272             # --- 4. 10Hz 数据录制（从独立线程取帧，不阻塞） ---
273             if self.record_state == "RECORDING" and loop_counter % re
                cord_interval == 0:
274                 state_7d = self._get_state_7d()
275                 action_7d = self._get_action_7d()
276                 frame = self.camera.get_frame()
277
278                 if state_7d is not None and frame is not None:
279                     # 缩放到SmolVLA推荐分辨率
280                     frame_resized = cv2.resize(frame, (IMG_SIZE, IMG_
                SIZE))
281                     frame_rgb = cv2.cvtColor(frame_resized, cv2.COLOR
                _BGR2RGB)
282
283                     self.dataset.add_frame({
284                         "observation.images.top": frame_rgb,
285                         "observation.state": np.array(state_7d, dtype
                =np.float32),
286                         "action": np.array(action_7d, dtype=np.float3
                2),
287                         "task": self.current_task,
288                     })
289
290             # --- 5. 低频画面预览（每0.1秒一次，不影响控制） ---
291             now = time.perf_counter()
292             if now - last_time_for_preview > 0.1:
```

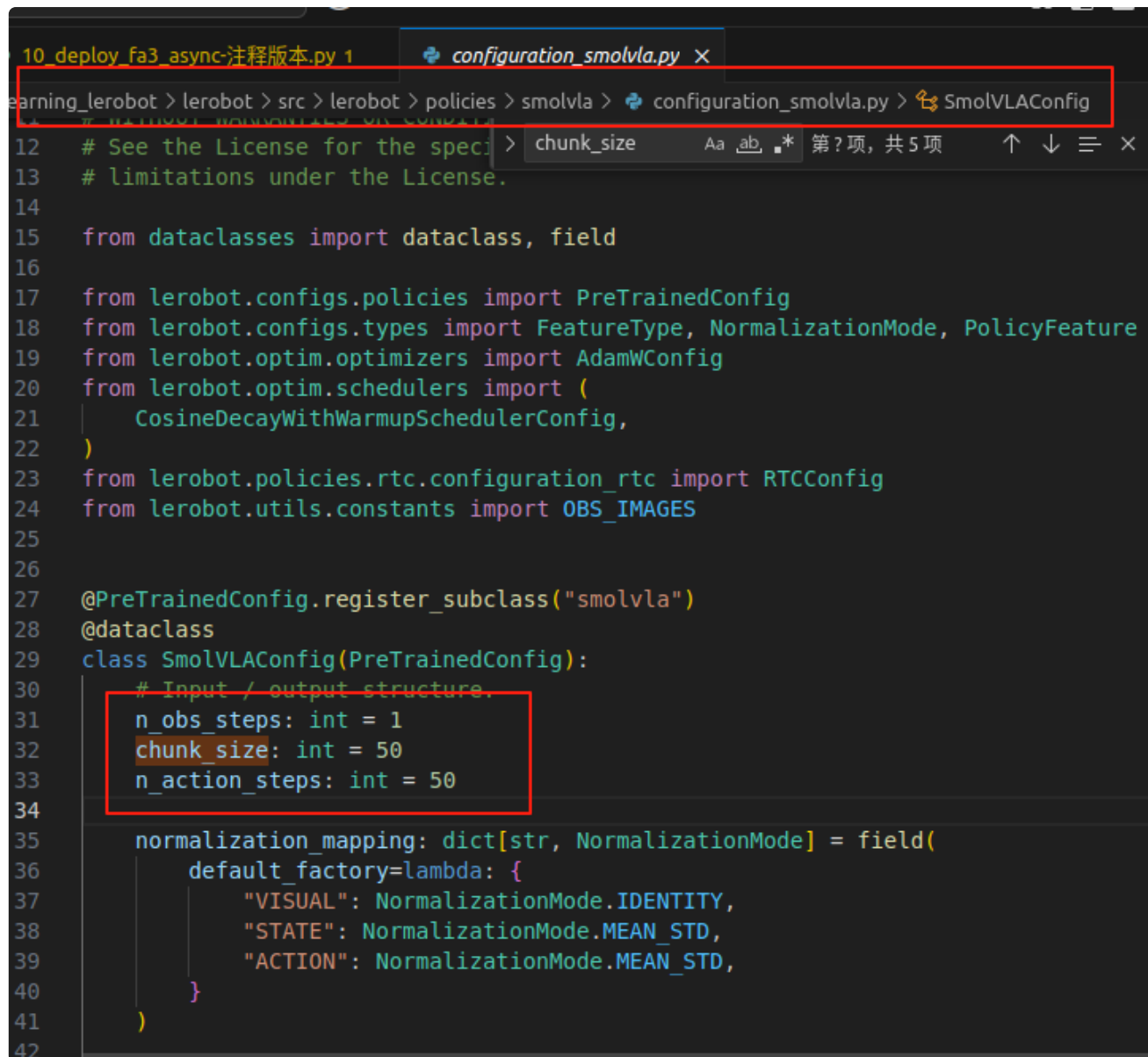
```
292         if now - last_display_time > 0.1:
293             frame = self.camera.get_frame()
294             if frame is not None:
295                 display = cv2.resize(frame, (480, 480))
296                 if self.record_state == "RECORDING":
297                     text = f"REC | {self.current_task[:35]}"
298                     color = (0, 0, 255)
299                 else:
300                     text = f"IDLE | {self.current_task[:35]}"
301                     color = (0, 255, 0)
302                 cv2.putText(display, text, (5, 25),
303                             cv2.FONT_HERSHEY_SIMPLEX, 0.5, color,
304
305             1)
306
307             cv2.imshow("FA3", display)
308             cv2.waitKey(1)
309             last_display_time = now
310
311             loop_counter += 1
312             elapsed = time.perf_counter() - t0
313             if elapsed < dt_control:
314                 time.sleep(dt_control - elapsed)
315
316     except KeyboardInterrupt:
317         pass
318     finally:
319         print("\n正在清理...")
320         listener.stop()
321         self.robot.ServoMoveEnd()
322         time.sleep(0.5)
323         self.robot.CloseRPC()
324         self.camera.stop()
325         cv2.destroyAllWindows()
326         if self.record_state in ("RECORDING", "SAVE_PENDING"):
327             self.dataset.save_episode()
328         self.dataset.finalize()
```

```
326         print(f"✅ 完成!")
327         self._print_summary()
328         print(f"路径: {self.dataset.root}")
329
330
331     if __name__ == "__main__":
332         app = FA3MultiTaskRecorder(
333             ip="192.168.58.2",
334             cam_idx=2,
335             repo_id="local/fa3_smolvla_multitask",
336         )
337         app.run()
```

## 5.对smolvla模型的输出动作序列的步数有疑问

### ( 1 ) 模型推理输出的动作块到底是多少步？怎么设定？

- 这个输出动作块的步数由训练决定，在训练时可以设置动作块的大小，训练时设置多少，模型在推理时就会输出多少动作步
- smolvla在使用lerobot的官方训练脚本时，官方默认的chunk\_size参数是50。



```
10_deploy_fa3_async-注释版本.py 1 configuration_smolvla.py X
learning_lerobot > lerobot > src > lerobot > policies > smolvla > configuration_smolvla.py > SmolVLAConfig
12 # See the License for the specific language governing rights and
13 # limitations under the License.
14
15 from dataclasses import dataclass, field
16
17 from lerobot.configs.policies import PreTrainedConfig
18 from lerobot.configs.types import FeatureType, NormalizationMode, PolicyFeature
19 from lerobot.optim.optimizers import AdamWConfig
20 from lerobot.optim.schedulers import (
21     CosineDecayWithWarmupSchedulerConfig,
22 )
23 from lerobot.policies.rtc.configuration_rtc import RTCCConfig
24 from lerobot.utils.constants import OBS_IMAGES
25
26
27 @PreTrainedConfig.register_subclass("smolvla")
28 @dataclass
29 class SmolVLAConfig(PreTrainedConfig):
30     # Input / output structure.
31     n_obs_steps: int = 1
32     chunk_size: int = 50
33     n_action_steps: int = 50
34
35     normalization_mapping: dict[str, NormalizationMode] = field(
36         default_factory=lambda: {
37             "VISUAL": NormalizationMode.IDENTITY,
38             "STATE": NormalizationMode.MEAN_STD,
39             "ACTION": NormalizationMode.MEAN_STD,
40         }
41     )
42
```

( 2 ) 在调用select\_action()时，输出的为什么是一个动作？

```
# -----  
# 3. SmolVLA 推理  
# -----  
with torch.no_grad():  
    # select_action 内部做了这件事：  
    # 1. 把 50 步存到内部缓存  
    # 2. 每次只返回 1 步给你  
    # action = policy.select_action(obs) # 返回 [1,7]  
    # 下次再调用，直接从缓存弹出下 1 步  
    # 直到 50 步用完，才会重新推理  
    action = self.policy.select_action([obs_processed])  
    print(f"模型输出的动作的形状: {action.shape}")
```

## 6.踩坑！微调训练命令中训练模型的加载方式的影响（详情见三、微调训练的4）

## 7.增量训练的一些坑！！！！

### （1）问题所在

我计划增加模型的空间泛化能力，想重新采集新数据，把新旧数据集混合之后，使用增量训练，但是训练的效果很差，新旧都不会了。

### （2）原因分析

最可能的原因是：从v1模型继续训练时，数据集的归一化统计量（mean/std）发生了变化。

#### 根因解释

当你用v1模型的checkpoint作为pretrained\_path时：



▼ Plain Text

```
1  v1模型训练时：30条数据 → 计算mean/std → 用这组统计量归一化 → 训练
2  v2继续训练时：50条数据 → 重新计算mean/std → 用新的统计量归一化 → 训练
3
```

新数据集包含了更大空间范围的数据，关节角度的mean和std都变了。这意味着：

- **v1模型的权重是基于旧的归一化参数学习的**
- 但现在输入的数据用了新的归一化参数
- 对模型来说，**同样的物理位置，归一化后的数值变了**，之前学到的映射关系全部失效

**这就是为什么不仅新位置抓不到，连基础位置也退化了。**

### ( 3 ) 验证分析

▼ Plain Text

```
1  用你的脚本统计输出结果：
2  State 统计量对比：
3      维度 |      旧mean      旧std |      新mean      新std |      mean差      std比
4      J1 |    -133.38    11.36 |    -133.73    11.87 |      -0.34      1.04x
5      J2 |     -72.28    14.76 |     -72.90    15.39 |      -0.62      1.04x
6      J3 |      70.10    20.58 |      70.84    21.97 |       0.74      1.07x
7      J4 |     -88.28    10.22 |     -87.71    10.77 |       0.56      1.05x
8      J5 |     -88.01     0.10 |     -88.23     0.50 |      -0.22      4.84x
9      J6 |    -128.49    11.37 |    -127.13    11.99 |       1.35      1.05x
10     Grip |      0.47     0.50 |      0.47     0.50 |      -0.00      1.00x
11
12  Action 统计量对比：
13     维度 |      旧mean      旧std |      新mean      新std |      mean差      std比
14     J1 |    -133.38    11.36 |    -133.73    11.87 |      -0.34      1.04x
15     J2 |     -72.28    14.76 |     -72.90    15.39 |      -0.62      1.04x
16     J3 |      70.10    20.58 |      70.84    21.97 |       0.74      1.07x
17     J4 |     -88.28    10.22 |     -87.71    10.77 |       0.56      1.05x
18     J5 |     -88.01     0.10 |     -88.23     0.50 |      -0.22      4.84x
19     J6 |    -128.49    11.37 |    -127.13    11.99 |       1.35      1.05x
20     Grip |      0.49     0.50 |      0.49     0.50 |       0.00      1.00x
```

分析结果

**J5的std变化了4.84倍！这就是问题的根源。**

具体影响:

|    | ▼                          | Plain Text                    |
|----|----------------------------|-------------------------------|
| 1  | J5维度：                      |                               |
| 2  | 旧数据集：mean=-88.01, std=0.10 | (J5几乎不动，范围极小)                 |
| 3  | 新数据集：mean=-88.23, std=0.50 | (泛化数据让J5有了更大变化)               |
| 4  |                            |                               |
| 5  | 影响：                        |                               |
| 6  | v1模型中，J5偏移0.1°             | 就是一个很大的归一化值： $0.1/0.10 = 1.0$ |
| 7  | v2训练时，同样偏移0.1°             | 的归一化值变成： $0.1/0.50 = 0.2$     |
| 8  |                            |                               |
| 9  | 模型权重是按旧的比例学习的，现在比例缩小了5倍    |                               |
| 10 | → J5维度的输出被压缩了5倍，控制精度严重下降   |                               |
| 11 | → 连带影响其他关节的协调运动，整体策略崩溃     |                               |

虽然其他关节的变化很小（1.04-1.07倍），但J5这一个维度4.84倍的变化就足以让整个模型失效。

为什么J5变化这么大

你的旧数据集中J5几乎是固定的（std=0.10°，基本不动），说明你在固定场景采集时J5关节变化极小。但泛化数据中物体位置变化更大，机械臂姿态变化增加，J5开始有明显运动。

## 8.消费频率调高后，运动抖动并且动作回退的现象

- 是因为消费频率调高后，出现了空队列，时间集成机制失效，chunk与chunk之间没办法平滑过渡了！

- 详细分析在：**12.最终使用版本的异步推理脚本理解**

## 五、第一次微调训练实机部署记录

### 5.1 第一次微调smolvla并部署到法奥真机上的效果

#### ( 1 ) 直接部署效果

- 1.机械臂运动非常抖动！！运动也很快！！完全不能直接完成抓取任务！
- 2.模型不能正确预测夹爪的开合
- 3.机械臂的大的方向是对的，**也就是模型学习到了这个指令是要去抓玩具鸭子，然后把鸭子放到纸杯**，但是具体的动作没有学会，具体表现就是抓不到东西，来回的调整，也放不进去

#### ( 2 ) 原因分析

##### 1.机械臂非常抖动的原因

- 我的异步推理脚本，设置的控制频率是100hz，而模型输出的每一动作步之间的关节角度插值我虽有所限制，限制的是2度，但是对于这个控制频率，这个角度还是太大了，导致继续要在短时间内完成很大的动作。这就导致了机械臂的剧烈抖动
- **问题解决：**
  - **1.我把控制频率降低为了20hz**
  - **2.单步角度变化限制在了0.6度。**
  - **3.增加了单步之间低通滤波（平滑指数）**

- 目前这个参数看来，机械臂的运动就没有那么抖动了（但是也算不上丝滑）

## 2. 夹爪开合预测有问题

- 原因是我在数据采集时：
  - 1. 把夹爪开、闭的状态和真实情况弄反了
  - 2. 而且模型学习的是连续的夹爪控制，不是二值化，可能会.....

## 3. 模型成功率很低（0%）

- 模型根本抓不到目标物体

## （3）第一次微调数据采集的情况（有问题）

### 1. 相机位置：

- 首先我数据采集只有一个第三视角（我只有一个相机），从上往下拍，效果如我上传的图片所示。  
（这个视角我其觉得会有点问题，因为从这视角来看，有些帧机械臂自身关节会有遮挡，不知道这个影响大不大。（但是这种遮挡是不可避免的，总会有一些角度会有遮挡））
- 存在问题：
  - 1. 相机离操作平台太远了，模型可能看不清楚
  - 2. 视角的设置会导致机械臂本体有较大的遮挡，而且夹爪有时候会对抓取物体有遮挡。（这也是为什么一般都会采用多视角输入，补全输入模型的信息）——这就导致模型输入的视觉信息有限（缺失）

### 2. 采集方案：

- A. 基础场景（20组）：鸭子在工作台前中间区域随机摆放（包含旋转），纸杯在工作台右侧区域随机摆放，都是小范围的移动大概在5~17cm的移动范围。

- B. 空间大范围泛化（20组）：鸭子遍历工作台9个区域（即四边形的左上/右上/左下/右下，以及每条边的中间）；纸杯同样遍历不同区域，且与鸭子不同侧。
- C. 干扰物品（10组）：加入盒子、笔等干扰物，干扰物放在鸭子和纸杯旁边。一共采集了50组数据。
- 存在问题：
  - 数据量太小，空间变化太大，在这么小的数据量中，想让模型学到这么大的空间变化，很难。这样看下来我每个大位置给模型学习的有效数据就只有2~3条，而smolvla又是一个预训练比较小的模型，所以这么小的数据量，模型可能根本学不会！！——我觉得这也直接导致了模型实际部署的效果很差，成功率非常低，模型确实没有学会（只有大的方向对了，但是做不到正确抓取和放置，这些比较细的操作还没学会，每个位置给模型学习的数据太少了）——后续多加数据可能会好一些（我猜测）——太贪心啦
  - 需要先让模型学会基础的操作在说泛化的事情。

### 3.采集方式：

- 关于机械臂的操作，我是用键盘控制的（法奥机械臂目前还不能主从遥控，拖动示教倒是可以，但是会把人拍进去），所以数据的动作可能会有一些不流畅，动作的分段和调整比较多。
- 存在问题：
  - 这也是导致模型部署时，一直来回调整的原因之一。

（4）

## 六、VLA模型持续学习与泛化增强方案

我的想法：通过继续训练，提高模型的空间泛化能力、以及接受新任务的能力。

### 1.核心原则

- 在已有模型基础上继续训练是完全可行的，但需要遵循一个关键原则：
  - **每次训练都必须混合旧数据和新数据一起训练**，而不是只用新数据训练。
  - 如果只用新数据训练，模型会发生灾难性遗忘——忘掉之前学会的东西。
- **降低学习率是防止遗忘的最关键手段。**

### 2.具体实施方案

#### 2.1 采集方案总览

▼

Bash

```
1 阶段1（已完成）： 30条固定场景数据 → 训练模型v1 → 85%成功率
2      ↓
3 阶段2（空间泛化）： 旧30条 + 新20条泛化数据 → 混合训练模型v2
4      ↓
5 阶段3（新任务）： 旧50条 + 新30条新任务数据 → 混合训练模型v3
```

2.2 阶段二：提升空间泛化能力的数据采集方案

▼

Bash

```
1  

|        |       |    |
|--------|-------|----|
| 工作台俯视图 |       |    |
| A1     | A2    | A3 |
| A4     | [原区域] | A5 |
| A6     | A7    | A8 |
| <hr/>  |       |    |
| 机械臂基座  |       |    |


9
10
11
12
13
14 A1-A8：新的鸭子摆放区域
15 纸杯也在不同位置变化
```



| 编号    | 数量 | 鸭子位置      | 纸杯位置   | 说明     |
|-------|----|-----------|--------|--------|
| 1-5   | 5条 | 偏左10-15cm | 固定     | 左侧泛化   |
| 6-10  | 5条 | 偏右10-15cm | 固定     | 右侧泛化   |
| 11-15 | 5条 | 偏前10-15cm | 固定     | 前方泛化   |
| 16-18 | 3条 | 中心区域      | 偏左10cm | 杯子位置变化 |
| 19-20 | 2条 | 中心区域      | 偏右10cm | 杯子位置变化 |

总计20条新数据

## 2.3 数据集融合的方法（脚本）

不要在旧数据集上追加！ 创建一个新的混合数据集

▼ merge\_datasets.pyPython |

```
1  #!/usr/bin/env python3
2  # merge_datasets.py
3  """将多个数据集合并为一个混合数据集用于训练"""
4
5  import torch
6  import numpy as np
7  from pathlib import Path
8  from lerobot.datasets.lerobot_dataset import LeRobotDataset
9  from tqdm import tqdm
10
11
12  def merge_datasets(dataset_paths: list, output_repo_id: str):
13      """
14      合并多个数据集为一个新数据集
15
16      参数:
17          dataset_paths: 要合并的数据集路径列表
18          output_repo_id: 输出数据集的repo_id
19      """
20      # 加载第一个数据集获取features定义
21      first_ds = LeRobotDataset(dataset_paths[0], video_backend="pyav")
22
23      # 创建新数据集
24      new_ds = LeRobotDataset.create(
25          repo_id=output_repo_id,
26          fps=first_ds.fps,
27          robot_type=first_ds.meta.robot_type,
28          features=first_ds.meta.features.copy(),
29          use_videos=True,
30      )
31
32      total_episodes = 0
33
34      for ds_path in dataset_paths:
```

```
35     print(f"\n处理数据集: {ds_path}")
36     ds = LeRobotDataset(ds_path, video_backend="pyav")
37     print(f"  包含 {ds.num_episodes} 个episode, {len(ds)} 帧")
38
39     # 按episode遍历
40     current_ep = -1
41     for i in tqdm(range(len(ds)), desc=f"  合并中"):
42         item = ds[i]
43         ep_idx = item["episode_index"].item()
44
45         # 新episode开始时, 保存上一个
46         if ep_idx != current_ep:
47             if current_ep >= 0:
48                 new_ds.save_episode()
49                 current_ep = ep_idx
50
51         # 处理图像格式
52         image = item["observation.images.top"]
53         if isinstance(image, torch.Tensor):
54             if image.shape[0] == 3:
55                 image = image.permute(1, 2, 0).numpy()
56             else:
57                 image = image.numpy()
58         if image.dtype != np.uint8:
59             image = (image * 255).clip(0, 255).astype(np.uint8)
60
61         new_ds.add_frame({
62             "observation.images.top": image,
63             "observation.state": item["observation.state"].clone(),
64             "action": item["action"].clone(),
65             "task": item["task"],
66         })
67
68     # 保存最后一个episode
69     new_ds.save_episode()
```

```
69         new_ds.save_episode( )
70         total_episodes += ds.num_episodes
71
72     new_ds.finalize()
73
74     print(f"\n{'='*60}")
75     print(f"✅ 合并完成!")
76     print(f"   数据集数量: {len(dataset_paths)}")
77     print(f"   总episode数: {total_episodes}")
78     print(f"   输出路径: {output_repo_id}")
79     print(f"{'='*60}")
80
81
82     if __name__ == "__main__":
83         merge_datasets(
84             dataset_paths=[
85                 # 旧数据集 (30条基础数据)
86                 "/home/zjs/.cache/huggingface/lerobot/local/fa3_smolvla_onetas
k_v2",
87                 # 新数据集 (20条泛化数据)
88                 "/home/zjs/.cache/huggingface/lerobot/local/fa3_smolvla_onetas
k_v2_spatial",
89             ],
90             output_repo_id="local/fa3_smolvla_onetask_v2_merged"
91         )
```

## 2.4 继续训练的命令（更小的学习率是关键）

**关键：**从v1模型继续训练，而不是从SmolVLA base重新训练



Python

```
1 python src/lerobot/scripts/lerobot_train.py \  
2     --policy.type=smolvla \  
3     --policy.pretrained_path=/home/zjs/你的v1模型路径/checkpoints/best/pretrai  
ned_model \  
4     --dataset.repo_id=/home/zjs/.cache/huggingface/lerobot/local/fa3_smolvla  
_onetask_v2_merged \  
5     --dataset.video_backend=pyav \  
6     --output_dir=/home/zjs/outputs/fa3_smolvla_v2_spatial \  
7     --batch_size=32 \  
8     --steps=6000 \  
9     --save_freq=1500 \  
10    --log_freq=50 \  
11    --policy.device=cuda \  
12    --policy.optimizer_lr=2e-5 \  
13    --policy.scheduler_warmup_steps=300 \  
14    --policy.scheduler_decay_steps=6000 \  
15    --policy.scheduler_decay_lr=5e-7 \  
16    --wandb.enable=true \  
17    --wandb.project=fa3_smolvla_v2_spatial
```

**和第一次训练的关键区别：**

| 参数              | 第一次 (从base)          | 继续训练 (从v1)          |
|-----------------|----------------------|---------------------|
| pretrained_path | lerobot/smolvla_base | 你的v1模型路径            |
| optimizer_lr    | 5e-5                 | 2e-5 (降低一半)         |
| steps           | 8000                 | 6000 (数据量增加但不需要太多步) |
| warmup_steps    | 400                  | 300                 |
| decay_lr        | 1e-6                 | 5e-7 (更小的最终学习率)     |

降低学习率是防止遗忘的最关键手段。

## 2.5 阶段三：加入新任务

### ( 1 ) 数据采集方案

假设新任务是 "把零食包放进盒子里"：

| 编号    | 数量  | 说明                 |
|-------|-----|--------------------|
| 1-15  | 15条 | 固定场景 (零食和盒子位置轻微变化) |
| 16-25 | 10条 | 轻微泛化 (位置变化稍大)      |
| 26-30 | 5条  | 加入干扰物品             |

采集时使用你脚本中已有的Task 2指令组 (按TAB切换)。

### ( 2 ) 合并新旧数据集

```
▼ Python
1 merge_datasets(
2     dataset_paths=[
3         # 任务1旧数据 (30条基础 + 20条泛化 = 50条)
4         "local/fa3_smolvla_onetask_v2_merged",
5         # 任务2新数据 (30条)
6         "local/fa3_smolvla_task2_snack",
7     ],
8     output_repo_id="local/fa3_smolvla_multitask_all"
9 )
```

### ( 3 ) 继续训练命令

多任务训练时学习率可以比空间泛化时稍高一点 (  $3e-5$  ) , 因为要学习新的策略。





Python

```
1 python src/lerobot/scripts/lerobot_train.py \  
2     --policy.type=smolvla \  
3     --policy.pretrained_path=/home/zjs/你的v2模型路径/checkpoints/best/pretrain  
ned_model \  
4     --dataset.repo_id=local/fa3_smolvla_multitask_all \  
5     --dataset.video_backend=pyav \  
6     --output_dir=/home/zjs/outputs/fa3_smolvla_v3_multitask \  
7     --batch_size=32 \  
8     --steps=10000 \  
9     --save_freq=2000 \  
10    --log_freq=50 \  
11    --policy.device=cuda \  
12    --policy.optimizer_lr=3e-5 \  
13    --policy.scheduler_warmup_steps=500 \  
14    --policy.scheduler_decay_steps=10000 \  
15    --policy.scheduler_decay_lr=5e-7 \  
16    --wandb.enable=true \  
17    --wandb.project=fa3_smolvla_v3_multitask
```

### 3.防止模型遗忘的核心策略

| 策略             | 具体做法                      | 重要程度  |
|----------------|---------------------------|-------|
| 混合训练           | 每次训练都包含所有旧数据              | ★★★★★ |
| 降低学习率          | 继续训练时lr降为首次的40-60%        | ★★★★  |
| 从已训模型继续        | pretrained_path指向上一版模型    | ★★★★  |
| 控制训练步数         | 不要过度训练，关注loss是否回升         | ★★★   |
| 保留多个checkpoint | save_freq设小，选最优checkpoint | ★★★   |

## 4.训练后的验证流程

▼ Python

```
1  1. 先测旧任务旧场景（确认没遗忘）
2    → 果成功率明显下降，说明遗忘了，需要降低lr或增加旧数据
3
4  2. 再测旧任务新场景（确认泛化提升）
5    → 果新场景成功率提升，说明泛化数据有效
6
7  3. 最后测新任务（果加了新任务）
8    → 确认新任务能做，且不影响旧任务
```

## 5.完整的继续训练路线图

